

Electoral Equilibrium

Task Instructions & Hour Ledger

320 Hours · 8 Weeks · 40 hrs/week

Research Assistant: Juhi Damley Supervising Mentor: Prof. Gaston Espinosa

Claremont McKenna College Summer 2026

The Big Idea

Electoral forecasting is static. Polling aggregators tell you where the race stands today but cannot answer: *“If **X** happens tomorrow, which voter groups must shift for the Democrat to still win?”*

Electoral Equilibrium answers in real time. Select a party (Democrat or Republican), type any hypothetical event — a scandal, an assassination, an October Surprise — and the app immediately shows how that party’s coalition of ten key voter groups (African Americans, Latinos, Asians, Seculars, Jews, Muslims, Women, Catholics, Protestants, Evangelicals) must rebalance, and the probability of achieving it.

Built with a fine-tuned Mistral 7B LLM, a mean-variance stochastic optimizer, and a two-stage NLP pipeline drawing on social media bio inference — grounded in two decades of Prof. Espinosa’s research on religion, race, and presidential voting.

How to Use This Document

Each task entry shows: a concrete deliverable, the files it touches, and its hour budget. Hours are set conservatively — if a task runs short, bank the time and use it as flex on the next heavy day. Every week closes with a **pull request** to main and a **brief weekly supervision check-in** with Prof. Espinosa.

Symbol	Meaning	Note
[N hrs]	Hour budget for this task	Running total per day = 8 hrs
☐	Checkbox to mark complete	Print and tick off
HPC	Run on CMC HPC cluster	Submit batch job, don’t block locally
M5	Run locally on MacBook Pro	Fast iteration / experimentation
PR	Pull request to main	Required before week closes

Hour Summary by Week

Week	Focus	Hours
1	Infrastructure & Artifact Contracts	40
2	Historical Data Pipeline	40
3	Baseline Voter Portfolio	40
4	RoBERTa Pipeline & Fine-Tune Dataset	40
5	LLM Fine-Tuning & Inference Endpoint	40
6	Stochastic Optimization & Monte Carlo	40
7	Web Application	40
8	Hardening, Validation & Deliverables	40
Total		320

AI Research Assistant Setup (Pre-Week — 4 hrs)

Do this before the first day of work. The goal is a 24/7 AI research assistant running on the Raspberry Pi 5 that you can message from Telegram or WhatsApp at any time. Routine queries (paper summaries, quick code questions, daily briefings) route to **Gemini Pro** for free. Deep architectural work routes to **Claude** on demand.

Component	Role	Cost
OpenClaw on Raspberry Pi 5	Gateway, memory, scheduling	Free (open-source)
Google Gemini Pro (AI Studio API)	Lightweight daily queries	Free tier / Pro sub
Anthropic Claude API	Deep reasoning and code	Pay per token
Telegram or WhatsApp	Chat interface	Free

Day 0 (Sun — Environment Setup: OpenClaw, Syncthing & Pi NPU (pre-work, 6 hrs))

□ **Task:** Apply for student programs and free tiers **before anything else**. (i) **GitHub Student Developer Pack** at education.github.com/pack — unlocks JetBrains IDEs, Namecheap domain (.me free for 1 year), Canva Pro, and more. Takes minutes. (ii) **Social media access — entirely free, no subscription needed.** Sign up for **Bluesky** at bsky.app (primary live collection, AT Protocol firehose, no cap) and **Apify** at apify.com (secondary live collection, free tier 500 results/run, covers the X demographic profile without any API cost). The bulk of historical Twitter data comes from free public archives downloaded in Week 4: 56M Kavanaugh tweets, 2020 election datasets, MeToo tweets, and more. **Do not sign up for the X Basic API (\$100/month) — the archives are superior for historical shocks and Bluesky/Apify covers live collection at zero cost.** (iii) **Meta Content Library** at developers.facebook.com/docs/content-library — free academic access; takes 2–4 weeks. Apply today. (iv) **Modal account** at modal.com — \$30/month free credit; at Mistral 7B on an A100 (approximately \$0.0003 per call), covers around 100,000 inference calls for free. Sign up immediately. [1 hrs]

□ **Task:** SSH into the Raspberry Pi 5. Install OpenClaw: `curl -fsSL https://openclaw.ai/install.sh | bash`. Run `openclaw onboard` and follow the wizard. Select Telegram or WhatsApp as the primary channel. Configure Gemini as default model and Claude Sonnet as the `@claude` route. Store both API keys in `/etc/environment`. Docker-containerise OpenClaw for security. [1 hrs]

□ **Task:** Install **Syncthing** on the four local machines only (M5, Intel Mac, Pi). **Do not install Syncthing on the HPC** — HPC nodes kill background daemons and a Syncthing daemon will be terminated silently, starving any SLURM jobs that depend on it. Create a shared `electoral-sync/` folder synced across the four local devices. Subdirectories: `rawdata/` (collected posts and articles), `artifacts/` (pipeline outputs), `data/finetune/` (training data). Test by writing a file on the Intel Mac and confirming it appears on the M5 within 30 seconds. HPC data transfer uses explicit `rsync/scp` before each job submission. [1 hrs]

□ **Task: Compile the Hailo HEF now in Week 0 — not Week 4.** NPU compilation to custom silicon routinely causes driver mismatches, ONNX graph translation failures, and quantization errors. This is a multi-day risk if hit in Week 4. Eliminate it now. (i) Install the Hailo Dataflow Compiler and SDK: `pip install hailo-sdk-client hailo-model-zoo onnx sentence-transformers`. Verify: `hailortcli fw-control identify` should return device info. (ii) Export all-MiniLM-L6-v2 to ONNX. (iii) Run `hailo optimize minilm.onnx --hw-arch hailo8l`. (iv) Run `hailo compile --hw-arch hailo8l -o models/minilm.hef`. (v) Verify: `from hailo_platform import HEF; HEF("models/minilm.hef")`. **CPU fallback (mandatory):** if compilation fails for any reason, set `"pi_npu_enabled": false` in `configs/base.json`. The bio classifier runs on Pi CPU at ~20ms/bio instead of sub-ms — slower but correct. The rest of the pipeline is unchanged. Week 4 must not be blocked on NPU compilation under any circumstances. [3 hrs]

□ **Task:** Set up the **Intel Mac** as the social media collection machine. Configure both free collectors:

- **Bluesky:** install `atproto` (`pip install atproto`). Test `app.bsky.feed.searchPosts` with the Ayatollah keywords. Confirm `author` `description` and `langs` fields are returned.
- **Apify:** install `apify-client` (`pip install apify-client`). Set the API token in `.env`. Test the X Scraper actor with the Ayatollah keywords. Confirm output matches the same `posts.jsonl` schema as `BlueSkyCollector`.

Set up a **launchd plist** so both collectors restart on reboot. `BlueSkyCollector` runs continuously (no cap). `ApifyCollector` is triggered per-shock (500 results/run free tier). Both write to `rawdata/social/{platform}/{shock_id}/posts.jsonl`. **Do not set up the X Basic API — it is not used in this project.** [2 hrs]

□ **Task:** Set up the **Intel Mac** for news scraping (previously Windows role): Python 3.11, jupyter, pandas, matplotlib, install Python 3.11, project dependencies, and Syncthing. Configure Syncthing to sync the shared folder. Confirm Jupyter can read files from `electoral-sync/rawdata/`. [1 hrs]

Parallel machine strategy for Weeks 4–5: Intel Mac runs Bluesky collector continuously and Apify per-shock. Pi NPU classifies bios in real time as posts arrive via Syncthing. Intel Mac runs the news scraper. HPC scores all collected text via SLURM array jobs. M5 does interactive development. No machine is idle.

Routing strategy for the 8 weeks: Use Gemini for paper summaries, quick questions, drafts, and anything you would normally Google. Switch to `@claude` for pipeline architecture, LaTeX writing, debugging complex logic, and anything requiring the full project context. This keeps your Claude Max usage reserved for real development sessions.

1 Week 0 — Infrastructure & Artifact Contracts (40 hrs)

Goal. By Friday, the pipeline skeleton exists, every stage has a typed artifact contract, and round-trip serialization tests pass. No economic logic is implemented this week.

End condition.

- ☐ `pytest` passes with zero failures.
- ☐ All payload classes serialize and deserialize without data loss.
- ☐ Smoke pipeline runs and writes one artifact file per stage.
- ☐ PR merged to `main`.

Day 1 (Mon — Repo Setup & Project Scaffolding)

☐ **Task:** Create the GitHub repository `electoral-equilibrium`. Initialize with `README.md`, `.gitignore` (Python), and `pyproject.toml`. Add project description, license (MIT), and a brief abstract in the README. [1 hrs]

☐ **Task:** Apply for Meta Content Library access now at `developers.facebook.com/docs/content-library`. Approval takes 2–4 weeks and is required for Week 4 Facebook data collection. Fill in the academic research use-case form: describe the electoral shock sentiment analysis project, cite Prof. Espinosa as faculty mentor, and list CMC as the institution. [1 hrs]

☐ **Task:** Set up the Python environment. Create a virtual environment, install core dependencies: `numpy`, `pandas`, `scipy`, `scikit-learn`, `cvxpy`, `pytest`, `black`, `ruff`. Pin all versions in `pyproject.toml`. [1 hrs]

☐ **Task:** Create the full directory skeleton from the appendix of the development plan. Every folder and empty `__init__.py` file should exist. Commit as `chore: scaffold repository structure`. [1 hrs]

☐ **Task:** Write `AGENTS.md`: contributor conventions, branch naming rules (`feature/weekN-description`), PR format, and the pull-request checklist from the development plan. [1 hrs]

☐ **Task:** Write `DOMAIN.md`: the specification for instantiating this pipeline in a new domain. This is the generalization contract — the document that makes the codebase a platform rather than a one-off tool. Required sections: (i) *What is a domain* — define stakeholder groups, loyalty metric, win condition, and equilibrium threshold; (ii) *Config fields to set* — list every field in `base.json` that is domain-specific; (iii) *Data to provide* — historical event-response dataset format, prediction market contract mapping file; (iv) *Fine-tuning schema* — confirm the JSONL format is domain-agnostic; (v) *Example: electoral vertical* — the current implementation as a worked example. The electoral build is instance 1 of a

general framework; `DOMAIN.md` makes that explicit from day one. [1 hrs]

□ **Task:** Create `electoral/core/types.py` with all type aliases: `Cycle`, `BlocId`, `Weight`, `ElasticityScore`, `ShockDelta`, `LoraRank`. Write a short docstring for each explaining the convention (format, valid range, rationale). [1 hrs]

□ **Task:** Create `configs/smoke.json` and `configs/base.json`. Smoke config uses two blocs, two cycles, and a tiny dataset path so the full pipeline can run end-to-end in under 10 seconds for testing. [1 hrs]

□ **Task:** Create `electoral/core/rng.py`: implement `derive_seed(tokens)` that hashes a list of string tokens into a reproducible integer seed, and `make_rng(seed)` that returns a seeded `np.random.Generator`. Write three unit tests in `tests/test_rng.py` confirming (i) identical inputs produce identical seeds, (ii) order matters, (iii) the generator produces identical draws across calls with the same seed. [1 hrs]

Day 2 (Tue — Schema Helpers & Artifact Envelope)

□ **Task:** Implement `electoral/core/schema.py` with all five validation helpers: `assert_required_keys`, `assert_unique`, `assert_sorted_unique`, `assert_valid_share`, `assert_shares_sum_to_one`. Every helper must raise `ValueError` with a message naming the class and field. No external dependencies. [3 hrs]

□ **Task:** Write `tests/test_schema.py`. For each helper, write at least two positive tests (valid inputs pass silently) and two negative tests (invalid inputs raise `ValueError` with an informative message). Run `pytest -v` and confirm all pass. [2 hrs]

□ **Task:** Create `electoral/core/io.py` implementing the Parquet + JSON artifact I/O layer (Option A). For the envelope (`stage`, `run_key`, `metadata`): `write_json(path, payload)` and `read_json(path)` using `indent=2`, `sort_keys=True`. For tabular data payloads (voter panel, scored posts, fine-tuning dataset): `write_parquet(path, df)` using `engine="pyarrow"`, `compression="snappy"` and `read_parquet(path)` returning a `pd.DataFrame`. Non-tabular payloads (optimizer output, simulation results) remain JSON. `write_artifact(path, envelope, df=None)` dispatches automatically. Write four tests: (i) JSON round-trip, (ii) Parquet round-trip with a 100-row fixture `DataFrame`, (iii) Parquet output is smaller than equivalent JSON (`assert parquet_size < json_size`), (iv) reading a non-existent path raises `FileNotFoundError`. [3 hrs]

□ **Task:** Define the artifact envelope in `electoral/artifacts.py`. Every artifact must contain `stage`, `run_key`, `metadata`, and `data` fields. Write `StageArtifact` as a frozen `dataclass` with `to_dict` and `from_dict`. [1 hrs]

Day 3 (Wed — Payload Classes (Part 1))

□ **Task:** Implement `VoterPanelData` in `electoral/artifacts.py`. Fields: `cycles`, `blocs`, `columns`, `n_rows`, `source`. Implement `to_dict`, `from_dict`, `validate` using schema helpers. Validation must check: `cycles` sorted unique, `blocs` unique, `n_rows` non-negative. [2 hrs]

□ **Task:** Implement `BaselinePortfolioData`. Fields: `method`, `party`, `weights`, `mu`, `target`. `party` is "democrat" or "republican" and must be validated as such. `weights[bloc_id]` is the optimized coalition weight in $[0, 1]$; must sum to 1.0. `mu[bloc_id]` is the historical mean within-bloc vote share *for the selected party* in $[0, 1]$. Validation: `weights` sum to 1.0 within `tol=1e-6`, all weight and `mu` values in $[0, 1]$, both dicts have identical key sets matching the 10 canonical bloc IDs, `party` is one of the two allowed values. [2 hrs]

□ **Task:** Implement `SentimentData`. Fields: `model`, `shocks`, `scores`. Validation must check: `shocks` unique, all score values in $[-1, 1]$, every bloc key in `scores` maps to a dict keyed by all shocks. [2 hrs]

□ **Task:** Implement `LLMFineTuneData`. Fields: `base_model`, `lora_rank`, `n_examples`, `cycles_used`, `adapter_path`. Validation: `lora_rank` positive, `n_examples` positive, `cycles_used` sorted unique. [2 hrs]

Day 4 (Thu — Payload Classes (Part 2) & Round-Trip Tests)

□ **Task:** Implement `ShockResponseData`. Fields: `shock`, `cycle`, `deltas`, `covariance`, `source`. `deltas[bloc_id]` is the LLM-estimated change in within-bloc Democratic vote share ($\Delta\mu_i$). `source` must be one of "llm_unified", "roberta_news_only", "roberta_social_only" — required for the three-way baseline comparison. Validation: all delta values finite and in $[-0.15, 0.15]$, `covariance` is square with side matching number of bloc keys, `source` non-empty string. [2 hrs]

□ **Task:** Implement `EquilibriumData`. Fields: `method`, `party`, `shock`, `weights`, `mu_shifted`, `feasible`, `target_met`, `target`. `weights[bloc_id]` is the rebalanced coalition weight $\tilde{w}_i$; `mu_shifted[bloc_id]` is the post-shock within-bloc vote share $\tilde{\mu}_i^{(P)}$ for the selected party. Both are required by the frontend to display coalition weight shifts and vote-share loyalty shifts for all 10 blocs. Validation: `weights` sum to 1.0, all weight and `mu_shifted` values in $[0, 1]$, `party` is valid, both dicts have identical key sets. [2 hrs]

□ **Task:** Implement `SimulationData`. Fields: `n_simulations`, `seed`, `win_probability`, `percentiles`. Validation: `win_probability` in $[0, 1]$, all percentile lists have exactly 5 values (p5, p25, p50, p75, p95) each in $[0, 1]$. [1 hrs]

□ **Task:** Implement `MetricsTablesData`. Fields: `tables`. Validation: `tables` keys non-empty strings, values JSON-serializable (call `json.dumps` inside `validate`). [1 hrs]

□ **Task:** Write `tests/test_artifact_roundtrip.py`. Implement the `assert_roundtrip(obj)` helper. Write one round-trip test per payload class using the minimal instances from the development plan. Add at least two negative tests: duplicate cycles in `VoterPanelData`, weights not summing to 1 in `BaselinePortfolioData`. [3 hrs]

Day 5 (Fri — Pipeline Skeleton, Smoke Run & PR)

□ **Task:** Implement `electoral/config.py`: `PipelineConfig` dataclass that loads `smoke.json` or `base.json`. Required fields: `run_key`, `seed`, `data_path`, `blocs`, `party` ("democrat" or "republican"), `target` (equilibrium threshold: Democrat ≈ 0.535 , Republican ≈ 0.520), `output_dir`. Validate that `party` is one of the two allowed values and that `target` is in (0.5, 0.7). Add "party": "democrat", "target": 0.535 to `smoke.json` and `base.json`. Implement `derive_seed` using `rng.py`. [2 hrs]

□ **Task:** Create `electoral/stages.py`: one stub function per stage (`build_voter_panel`, `build_baseline_portfolio`, `build_sentiment_data`, `build_llm_finetune`, `build_shock_response`, `build_optimization`, `run_simulations`). Each stub accepts `PipelineConfig`, constructs a minimal placeholder payload using the correct dataclass, wraps it in a `StageArtifact`, and writes it to disk via `io.write_artifact` (not `write_json` — tabular payloads go to Parquet). Every week replaces its stub with real logic. [2 hrs]

□ **Task:** Write a `Justfile` at the repo root (install just via `brew install just`). Required targets: `smoke` (runs pipeline on `configs/smoke.json`), `test` (runs `pytest tests/ -v`), `score` (submits RoBERTa SLURM array), `train` (submits HPC fine-tuning job), `deploy` (runs modal deploy `deploy/modal_app.py`), `sample` (submits `scripts/sample_archives.py` as a SLURM array to draw stratified samples from raw archives), `clean` (runs `scripts/clean_with_llm.py` locally on the sampled dataset using Gemini Pro), and `continuous` (runs `prefect run pipeline.py --config configs/continuous.json` for nightly incremental updates post-SRP). This saves repeated long commands and eliminates flag typos across 8 weeks of daily use. [1 hrs]

□ **Task:** Implement `electoral/core/io.py`: `write_artifact(path, envelope)` writes the JSON envelope; for tabular data payloads (panel, scored posts, fine-tuning dataset) writes the data as Parquet via `pandas.DataFrame.to_parquet()`. `read_artifact(path)` reads the envelope and auto-detects Parquet vs JSON for the data payload. Parquet is 5–10 $\times$ faster than JSON for large tabular data and smaller on disk (Syncthing benefit). Confirm round-trip: write a 1,000-row toy `DataFrame`, read it back, assert equality. [2 hrs]

□ **Task:** Replace `electoral/pipeline.py` with a **Prefect flow**. Install `prefect`. Convert each stage function into a `@prefect.task` with `retries=2` and `retry_delay_seconds=30`. Wrap the full pipeline in a `@prefect.flow`. Benefits: automatic retry on failure, parallel task execution where tasks are independent, a visual dashboard (`prefect server start`) showing

stage completion status, and persistent state so a crash restarts from the last successful stage. Run `prefect server start` locally and confirm the smoke pipeline appears in the dashboard. [2 hrs]

☐ **Task:** Weekly PR: open `feature/week1-artifact-contracts` against `main`. PR description must include: summary of what was implemented, the pre-submission checklist from `AGENTS.md`, and any design choices made. Run all tests locally and merge after they pass. [1 hrs]

☐ **Task:** Brief supervision check-in with Prof. Espinosa: confirm the canonical bloc names and equilibrium threshold $V_{eq} = 0.535$. This is a domain confirmation only — Juhi drives the agenda. Document decisions in `DECISIONS.md`. [1 hrs]

2 Week 1 — Historical Data Pipeline (40 hrs)

Goal. By Friday, the data kernel ingests raw survey exports from all five sources and produces a validated `VoterPanelData` artifact.

End condition.

- ☐ Panel passes all invariants: unique tuples, sorted, finite values in $[0, 1]$.
- ☐ Toy fixture (`toy_panel.csv`) runs through the full cleaning pipeline.
- ☐ All data tests pass.
- ☐ PR merged.

Day 1 (Mon — Data Source Audit & Fixture Creation)

☐ **Task:** Download and inspect raw data from all five sources: ARDA, GSS, Gallup, NEP, and Pew. For each source, document: available years, column names, demographic breakdowns, and any known data quality issues. Write findings in `rawdata/SOURCE_NOTES.md`. [3 hrs]

☐ **Task:** Create `tests/fixtures/toy_panel.csv`: a minimal 20-row CSV with columns `cycle`, `bloc`, `vote_share`, `turnout`, `source`. Cover all 10 blocs across two cycles (2016, 2020). Include one intentional dirty row (missing value, wrong type) to test cleaning logic. [2 hrs]

☐ **Task:** Identify and document the column name mapping for each real data source to the canonical panel schema. Store as `rawdata/column_maps.json`: a dict mapping source name to a dict of `{raw_col: canonical_col}`. [3 hrs]

Day 2 (Tue — Loaders & Panel Utilities)

☐ **Task:** Implement `electoral/data/loaders.py: load_csv_panel(path)` that reads a CSV with explicit dtypes (`cycle` as int, `bloc` as str, `vote_share` and `turnout` as float64). No date inference. No random sampling. Returns a `pd.DataFrame`. [2 hrs]

☐ **Task:** Implement `electoral/data/panel.py: validate_panel(df, required_cols, context)` that enforces all five invariants from the development plan. Each violated invariant must raise a `ValueError` naming the class and field. Write five unit tests — one per invariant. [3 hrs]

☐ **Task:** Implement source-specific loader wrappers in `electoral/data/loaders.py: load_arda, load_gss, load_gallup, load_nep, load_pew`. Each applies the correct column map from `column_maps.json` and returns a raw (uncleaned) `DataFrame`. [3 hrs]

Day 3 (Wed — Cleaning Logic)

□ **Task:** Implement `electoral/data/cleaning.py: clean_raw_panel(df)` that applies all required cleaning steps: coerce `cycle` to int YYYY, `bloc` to lowercase snake-case str, `vote_share` and `turnout` to float64, drop rows with missing required values (log how many dropped), sort by (`cycle`, `bloc`), raise on duplicate (`cycle`, `bloc`, `source`) tuples rather than silently aggregating. [3 hrs]

□ **Task:** Implement bloc name normalization in `cleaning.py: normalize_bloc(raw_name)` that maps source-specific demographic labels to the canonical bloc identifiers confirmed by Prof. Espinosa (e.g., "White Evangelical" → "evangelical"). Build the map from `DECISIONS.md`. Raise on unrecognized bloc names. [3 hrs]

□ **Task:** Write `tests/test_data_cleaning.py`. Test: (i) clean path with toy fixture produces correct dtypes and sorting, (ii) dirty row is dropped and logged, (iii) unrecognized bloc name raises informatively, (iv) duplicate tuple raises rather than silently aggregating. [2 hrs]

Day 4 (Thu — Data Kernel & Multi-Source Merge)

□ **Task:** Implement `electoral/kernels/data.py: build_voter_panel(config)` that loads all five sources, applies source-specific loaders, concatenates, cleans via `clean_raw_panel`, validates via `validate_panel`, constructs a `VoterPanelData` payload, and returns both the payload and the cleaned DataFrame. Wire into `stages.py` replacing the Week 1 stub. [3 hrs]

□ **Task:** Implement multi-source conflict resolution: when the same (`cycle`, `bloc`) pair appears in multiple sources with different `vote_share` values, compute the weighted average (weight = inverse standard error if available, else equal weight). Document the rule in `DECISIONS.md`. [3 hrs]

□ **Task:** Run the full data kernel on real data for all cycles 2000–2024. Inspect the output DataFrame. Log counts per source per cycle. Flag any bloc-cycle pairs with missing data across all sources. Document gaps in `rawdata/SOURCE_NOTES.md`. [2 hrs]

Day 5 (Fri — Integration, Visualization & PR)

□ **Task:** Write a data audit script `scripts/audit_panel.py` that loads the cleaned panel and prints: (i) coverage matrix (bloc × cycle showing which cells have data), (ii) vote-share summary statistics per bloc, (iii) any obvious outliers (vote share < 0.05 or > 0.95). Save output as `rawdata/audit_report.txt`. [2 hrs]

□ **Task:** Manually cross-check 10 spot values from the panel against the original source documents. Record the spot-check results in `rawdata/spot_check.md`. Flag any large discrepancies for the supervision check-in. [2 hrs]

□ **Task:** Weekly PR: `feature/week2-data-pipeline`. Ensure all tests pass, the audit report is committed, and the PR description summarizes any data quality decisions made. Merge after tests pass. [1 hrs]

□ **Task:** Brief supervision check-in with Prof. Espinosa: share the coverage matrix and get domain input on any missing bloc-cycle cells. Juhi decides and implements the imputation strategy; document in `DECISIONS.md`. [2 hrs]

□ **Task:** Implement the agreed imputation strategy (likely: linear interpolation for consecutive missing cycles, or carry-forward for isolated gaps) in `cleaning.py`. Add unit tests. [1 hrs]

3 Week 2 — Baseline Voter Portfolio (40 hrs)

Goal. By Friday, the ML pipeline produces a validated `BaselinePortfolioData` artifact representing the optimal steady-state voter equilibrium free of shocks.

End condition.

- ☐ Baseline weights sum to 1.0 and every weight is in $[0, 1]$.
- ☐ The portfolio achieves $\mathbf{w}^\top \boldsymbol{\mu}^{(P)} \geq V_{\text{eq}}^{(P)}$ on historical winning cycles for the configured party.
- ☐ Leave-one-cycle-out validation documented for both parties.
- ☐ PR merged.

Day 1 (Mon — Moment Estimation)

☐ **Task:** Implement `electoral/models/ml.baseline.py: estimate_moments(df, party)` that computes $\boldsymbol{\mu}^{(P)}$ (mean within-bloc vote share for party P across winning cycles) and $\boldsymbol{\Sigma}$ (empirical covariance across cycles). For `party="democrat"` use the `vote_share` column directly; for `party="republican"` use `1 - vote_share`. Define “winning cycle” as cycles where the selected party’s candidate exceeded 50% of the two-party vote share nationally — this is party-configurable. Apply a PSD safeguard to $\boldsymbol{\Sigma}$: add ϵI if the minimum eigenvalue is negative. [3 hrs]

☐ **Task:** Write helper `psd_repair(cov, eps=1e-6)` in `ml.baseline.py` that adds ϵI to a matrix until it is positive semi-definite. Write a unit test confirming the output is PSD for a known near-singular matrix. [2 hrs]

☐ **Task:** Log and inspect the estimated $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. Print bloc names alongside their mean vote shares. Cross-reference against Prof. Espinosa’s published research (e.g., Evangelicals near 24%). Flag any large discrepancies for the supervision check-in. [3 hrs]

Day 2 (Tue — CVXPY Baseline Optimization)

☐ **Task:** Implement `electoral/portfolios/cvx.py: solve_baseline(mu, cov, target)` that formulates and solves the constrained quadratic program: $\min_{\mathbf{w}} \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ subject to $\sum w_i = 1$, $w_i \geq 0$, $\mathbf{w}^\top \boldsymbol{\mu} \geq V_{\text{eq}}$. Use `cvxpy.Problem`. Return weights as a dict keyed by bloc name. Raise informatively if problem is infeasible. [3 hrs]

☐ **Task:** Implement solver fallback in `cvx.py`: if CVXPY returns infeasible, relax the equilibrium constraint by 1% increments and retry up to 5 times. Log each relaxation. If still infeasible after 5 retries, raise `ValueError` with the final constraint value attempted. [2 hrs]

☐ **Task:** Write `electoral/portfolios/weights.py: equal_weight_baseline(blocs)` and

`value_weight_baseline(df, blocs)` as simple benchmark alternatives to the CVXPY optimizer. These serve as sanity checks and paper baselines. [3 hrs]

Day 3 (Wed — Gaussian Process Classifier)

□ **Task:** Implement the **Gaussian Process** win-probability classifier in `ml_baseline.py` (replacing XGBoost; Option E). Use `sklearn.gaussian_process.GaussianProcessClassifier` with a composite kernel: `RBf(length_scale=1.0) + Matern(length_scale=1.0, nu=1.5)`. Feature matrix X : per-bloc vote shares, turnout rates, cycle index (numeric, so the GP sees temporal ordering). Target y : binary win. The GP provides calibrated posterior predictive probabilities and explicitly models temporal correlations — the 2020 cycle is weighted more heavily than 2000 when predicting 2024. Extract the posterior standard deviation per held-out cycle and store in `artifacts/baseline_loco.json` alongside accuracy. These uncertainty estimates feed the paper’s epistemic humility framing. [4 hrs]

□ **Task:** Write `tests/test_baseline.py`: (i) `estimate_moments` returns correct shapes, (ii) `psd_repair` output is PSD, (iii) `solve_baseline` weights sum to 1.0, (iv) GP classifier outputs calibrated probabilities in $[0, 1]$, (v) infeasible QP raises correctly. [4 hrs]

Day 4 (Thu — Kernel Integration & Validation)

□ **Task:** Implement `electoral/kernels/baseline.py`: `build_baseline_portfolio(config, panel_df)` that orchestrates moment estimation, CVXPY optimization, and payload construction. Wire into `stages.py` replacing the Week 1 stub. [3 hrs]

□ **Task:** Run leave-one-cycle-out (LOCO) validation: for each cycle c , train on all other cycles, predict win probability on c , compare to ground truth. Record accuracy in `artifacts/baseline_loco.json`. **Also** compare the GP’s predicted win probability against the IEM/PredictIt market-implied price at the time of that election (for cycles 2004+). If the model diverges from the market by more than 10 percentage points on a held-out cycle, investigate the moment estimates and constraint bounds before proceeding to Week 4 — a large divergence signals a calibration problem, not just model uncertainty. [4 hrs]

□ **Task:** Inspect the optimized weights. Cross-reference high-weight blocs against Prof. Espinosa’s published demographic research. Write a 1-page interpretation in `docs/baseline_interpretation.md`. [1 hrs]

Day 5 (Fri — Constraints, Edge Cases & PR)

□ **Task:** Implement `electoral/portfolios/constraints.py`: `ConstraintSpec` dataclass specifying simplex constraints plus any user-defined per-bloc bounds (e.g., Evangelicals must be at least 20%). Integrate into `solve_baseline`. [2 hrs]

□ **Task:** Add edge case handling: (i) single-bloc input, (ii) all blocs equal weight produces feasible solution, (iii) target > 1.0 raises immediately without calling solver. Add tests for each. [2 hrs]

□ **Task:** Weekly PR: `feature/week3-baseline-portfolio`. Include the LOCO validation results and interpretation doc in the PR. Merge after tests pass. [1 hrs]

□ **Task:** Brief supervision check-in with Prof. Espinosa: share the baseline weights and LOCO results. Get domain feedback on whether the demographic ordering aligns with his published research. Juhi decides on any constraint adjustments and documents in `DECISIONS.md`. [3 hrs]

□ **Task:** Implement `electoral/kernels/raking.py`: marginal calibration (iterative proportional fitting) as an optional post-processing step on the stratum weights from `build_constraint_spec`. The algorithm: (1) rescale race weights to sum to 1 matching voter panel race marginals; (2) rescale religion weights matching religion marginals; (3) rescale gender weights matching gender marginals; repeat until convergence (tolerance $< 10^{-6}$, typically < 10 iterations). Output the raked weights under key `"raked"` in `configs/layer_weights.json` alongside the additive `"additive"` weights. Log the per-iteration change and number of iterations to converge. This is the lightweight middle ground between the additive model and full MRP; it enforces joint consistency across all three marginals without requiring cross-tabulation data. The paper must compare additive vs. raked outputs. [4 hrs]

4 Week 3 — RoBERTa Pipeline, Social Media & Fine-Tuning Dataset (40 hrs)

Goal. By Friday, two parallel sentiment pipelines are running — a news corpus scraper and a four-platform social media collector. Both are scored with RoBERTa, social media sentiment is correlated with 14-day lagged favorability polls, and a unified fine-tuning dataset is assembled.

End condition.

- ☐ `SentimentData` and `SocialMediaSentimentData` artifacts validate and round-trip.
- ☐ At least 400 fine-tuning examples in `train.jsonl`, fusing both news and social media signals.
- ☐ Baseline-adjusted social sentiment computed for all platforms.
- ☐ `eval.jsonl` contains 2020 cycle examples only.
- ☐ PR merged.

Hard fallbacks — read before starting Week 4.

Meta Content Library: If not approved by Week 4 Day 1, do not wait. Activate the fallback immediately: scrape public Facebook page reaction counts (like, angry, sad, love) from major news pages as a valence-only signal. The `FacebookCollector` stub must work in both modes — full Content Library and reaction-count fallback — before Day 2 ends.

Social media collectors: Bluesky and Apify should both be running from Week 0. If Apify's free tier runs out for a given shock, reduce the per-run result count or skip that shock for Apify and rely on Bluesky alone. The historical archives (Kavanaugh, 2020 election, MeToo) are the primary source of Twitter-equivalent signal — live collection is supplementary.

Hailo NPU: If Week 0 compilation failed, confirm `pi_npu.enabled=false` in `configs/base.json`. The CPU fallback path must be running and tested before Day 3's SetFit training begins.

Day 1 (Mon — Shock Taxonomy, API Setup & News Scraper)

☐ **Task:** Build the shock event registry: `configs/shocks.json` listing all historical shock events 2000–2024. Each entry: `id` (slug), `date`, `category` (Security, Geopolitical, Moral/S-candal, Electoral Surprise), `description` (one sentence), `target.blocs`, `keywords`, and `archive_ids` (list of pre-collected dataset slugs that cover this event, or empty list if live collection only). Aim for at least 25 events. Mark the following shocks as archive-covered in `archive_ids`:

- `kavanaugh_2018`: `["kavanaugh"]` — 56M tweets
- `beto_cruz_debate_2018`: `["texas_debate_2018"]`
- `metoo_2017`: `["metoo"]`
- Any 2020 cycle events: `["election_2020_ieee", "election_2020_kaggle"]` as applicable

Use the Ayatollah assassination as the pilot/test case throughout this week.

[2 hrs]

□ **Task:** Verify all Week 0 social media setup is fully operational before writing any Week 4 code: (i) `BlueSkyCollector` returns posts with author `description` and `langs` fields for the Ayatollah keywords; (ii) `ApifyCollector` returns posts in the correct schema; (iii) the Pi bio server responds at `http://pi.local:9000/health` in either NPU or CPU fallback mode; (iv) Syncthing is syncing between Intel Mac and M5. Log the status of both collectors in `DECISIONS.md` with the date. [1 hrs]

□ **Task:** Compile `all-MiniLM-L6-v2` for the Pi AI HAT+ NPU. On the Pi, run `scripts/compile_hailo.py`: (i) load the model via `sentence-transformers`, (ii) export to ONNX with a dummy input of shape (1,128), (iii) run `hailo optimize minilm.onnx --hw-arch hailo8l`, (iv) run `hailo compile minilm.onnx --hw-arch hailo8l -o models/minilm.hef`. Verify the HEF loads: `from hailo_platform import HEF; HEF("models/minilm.hef")`. This step can run while the Intel Mac setup runs in parallel.

Then start the bio server: `uvicorn pi_bio_server:app --host 0.0.0.0 --port 9000`. **Hostname:** `pi.local` requires mDNS/Bonjour. If it does not resolve from the Intel Mac or M5, assign a static IP to the Pi in your router, add it to `/etc/hosts` on all machines, and update "pi_bio_server" in `configs/base.json`. Confirm reachability: `curl http://pi.local:9000/health`. [2 hrs]

Storage strategy — read before downloading anything.

The raw archives total several hundred gigabytes. Follow the **fetch, extract, discard** principle throughout:

- **Intel Mac** is the primary archive machine (has available local disk). Social media archives (Kavanaugh, 2020 election, Webhose, etc.) download here. After sampling, the full raw archive can be deleted — keep only the sampled JSONL (~200MB per archive).
- **3DLNews2:** download only the URL metadata file to the Intel Mac (small). Filter by date window and state *before* fetching any HTML. Run the HTML fetch-and-parse as a SLURM array job on HPC scratch so raw HTML is never written to local disk. `rsync` only the extracted `articles.jsonl` back to the Intel Mac.
- **Syncthing** syncs processed outputs only (sampled JSONL, cleaned JSONL, scored embeddings). Never configure Syncthing to sync raw archives.
- **M5** holds no raw archives — processed outputs only.

□ **Task:** Download all pre-collected archives and inspect their schemas. Save each to `data/archives/{slug}/` and document schema findings in the archives README before Day 2.

Tier 1 — full objects, directly usable:

- **Kavanaugh (56M, Sept–Oct 2018):** pushshift.io/kavanaugh-twitter-dataset/.

- **2020 Election — IEEE (3.5M, July–Aug 2020):** iee-dataport.org/open-access/us-november-2020-election-tweets-dataset.
- **2020 Election — Kaggle (Oct 15–Nov 4 2020):** kaggle.com/manchunhui/us-election-2020-tweets.
- **#MeToo tweets:** data.world/rdeeds/350k-metoo-tweets.
- **Congress tweets (ongoing JSON):** alexlitel.github.io/congresstweets/.
- **2018 Texas Debate (Beto vs Cruz):** digital.library.unt.edu/ark:/67531/metadc1259409/.
- **Polarization corpus (Kiran et al.):** gvrkiran.github.io/polarizationTwitter/. **High priority.** Add bio text + ideological label to data/bio_labels/labeled_bios.jsonl before SetFit training (Week 4 Day 3).
- **SMAPP NYU elections:** github.com/SMAPPNYU/twitter_elections_public_interest.
- **UW ResearchWorks (b88ba40a):** digital.lib.washington.edu/researchworks/items/b88ba40a-4001-4fb4-b960-2ea2e49d58a0. Verify content is political before loading.
- **USC Telegram 2024 election (leonardo-blas):** github.com/leonardo-blas/usc-tg-24-us-election and huggingface.co/datasets/leonardoblas/us_election_2024_telegram_distilled. No user bio field — apply Evangelical/conservative Protestant platform-level proxy to all posts. Use TelegramLoader.
- **Truth Social 2024 election (kashish-s):** github.com/kashish-s/TruthSocial_2024ElectionInitiative. Evangelical/conservative Protestant demographic proxy.

Tier 2 — scorer / demographic training:

- **Political tweets (kaushiksuresh147):** kaggle.com/datasets/kaushiksuresh147/political-tweets.
- **Political partisanship (lingshuhu):** kaggle.com/datasets/lingshuhu/political-partisanship-tweets.
- **Political tweets + gender (lingshuhu):** kaggle.com/datasets/lingshuhu/political-tweets-with-gender-info.
- **PoliticalTweets (HuggingFace):** huggingface.co/datasets/Jacobvs/PoliticalTweets.
- **US Politicians tweets (mrmorj):** kaggle.com/datasets/mrmorj/us-politicians-twitter-dataset.
- **TikTok 2024 election (gabbypinto):** github.com/gabbypinto/US2024PresElectionTikToks. Load caption text as text field; apply younger/secular/diverse proxy. Use TikTokLoader. No bio field available.

Format unconfirmed — check before downloading:

- **X 2024 election (sinking8):** github.com/sinking8/x-24-us-election. Visit repo and check: if full tweet objects, save to `data/archives/x_2024/` and add to Tier 1. If IDs only, skip (not usable without paid API).
- **US Pres Elections 2020 (echen102):** github.com/echen102/us-pres-elections-2020. Same check. Emily Chen's datasets are typically IDs — but check for any full-object sample files in the repo.

Bot filter — blocklist only:

- **Political Twitter bots (khanradcoder):** Kaggle: [khanradcoder/political-tweets-from-twitter-bots](https://www.kaggle.com/khanradcoder/political-tweets-from-twitter-bots). Extract bot user IDs into `data/archives/bot_blocklist/ids.txt`. **Do not use as training data.**

News archives — historical article text:

- **3DLNews2 (W&M NewsLab) — Tier 1, primary:** github.com/wm-news/3DLNews. 8M+ URLs from 14,000 local outlets, 1995–2024. **Do not fetch HTML yet.** Download the URL metadata file only and save to `data/archives/news/3dlnews/`. Filter by date window and state before fetching — this reduces the fetch set from millions to tens of thousands of URLs. Run the HTML fetch-and-parse as a SLURM array job on HPC scratch (see the storage strategy box above). Tag with `source="3dlnews"`.
- **Webhose free news dataset — Tier 2, secondary:** github.com/Webhose/free-news-datasets. Full article text pre-extracted, multi-outlet. Save to `data/archives/news/webhose/`. Tag with `source="webhose"`.

For each dataset, confirm presence of `text` (or `full_text`), `created_at`, and `lang`. Record all findings in the archives README before Day 2.

[5 hrs]

□ **Task:** Write `scripts/sample_archives.py` and submit it as a **HPC SLURM array job** to draw stratified samples from the raw archives. This runs *before* any RoBERTa scoring and reduces the corpus from hundreds of millions of posts to a tractable, representative sample.

Three strata per archive per shock event:

1. **Temporal:** sample proportionally by day across the shock window (e.g. 3 days before, day-of, 3 days after, 7 days after) so the full arc of public reaction is captured rather than just the peak day.
2. **Demographic:** after a fast keyword-based bio pass, sample proportionally by inferred bloc so all 10 groups are represented. Without this step, high-volume blocs (secular, evangelical) crowd out low-volume blocs (Jewish, Muslim).
3. **Sentiment extremity:** run a lightweight keyword sentiment score (VADER or similar) and oversample posts at the extremes (top and bottom 20% of the distribution). Extreme examples are more informative for fine-tuning than neutral ones.

Target output: ~200,000 posts per major archive dataset, ~5,000 per shock event per platform. Submit as a SLURM array job (`#SBATCH --array=0-N` where N is the number of

archive/shock combinations). Add `just sample` to the Justfile. [3 hrs]

□ **Task:** Write `scripts/clean_with_llm.py` and run it on the **M5** using a local open-weight model (Qwen2.5-7B or Mistral-7B-Instruct) (free tier: 1,000 requests/day) during the initial build phase. In continuous mode post-SRP, this script runs on the **Intel Mac** as part of the nightly launchd pipeline. Four cleaning operations:

1. **Off-topic detection.** For each post, prompt Gemini: *“Is this post about [shock event]? Reply yes or no.”* Drop all “no” responses. This removes posts that mention the shock keyword coincidentally (e.g. Kavanaugh baseball tweets during the confirmation hearing).
2. **Spam and low-information filtering.** Remove posts that survive the bot blacklist but add no signal: pure retweet text, copy-paste chain tweets, emoji-only posts, bare URL shares with no commentary.
3. **Text normalisation.** Expand political abbreviations (SCOTUS, POTUS, MAGA, FLOTUS), normalise hashtag formatting, strip URLs and @mentions so RoBERTa receives cleaner input without domain-specific noise.
4. **Cross-platform deduplication.** The same post sometimes appears in multiple archives. Deduplicate on `hash(user_id + text[:50])` before scoring.

Throughput: ~ 50 posts per Gemini prompt $\times$ 1,000 requests/day = 50,000 posts/day. For 200k posts per archive this takes 4 days. **Start the most important archives (Kavanaugh, 2020 election) during Weeks 2–3** while other components are being built, so cleaned data is ready for the Week 4 RoBERTa scorer. Add `just clean` to the Justfile. Log the discard rate per cleaning step in `data/archives/README.md`. [3 hrs]

□ **Task:** Implement `electoral/nlp/scrapper.py` on the **Intel Mac**: `scrape_articles(shock_id, sources, date_range)` for news sources (Christianity Today, CBN, Univision, NYT, WaPo, Fox). Strip HTML, extract plain text, save to the Synthing `rawdata/articles/` folder so the M5 can read results immediately. Enforce 100-word minimum, log discard rate, test on Ayatollah shock. [1 hrs]

□ **Task:** Implement `electoral/nlp/news_loader.py`: `NewsArchiveLoader` that normalises pre-collected news articles into the same `articles.jsonl` schema as the live scraper, so the RoBERTa scorer is blind to source.

Two sources, handled as subclasses or branches:

- **3DLNewsLoader:** reads URL entries from `data/archives/news/3dlnews/`; filters to the shock date window using `publication_date`; fetches and HTML-strips each article (same logic as `scraper.py`); optionally filters by `state` or outlet name for targeted demographic coverage; enforces 100-word minimum; tags `source="3dlnews"`. This is the primary historical news source — 14,000 local outlets covering the communities that constitute each of the 10 blocs.
- **WebhoseLoader:** reads pre-extracted text from `data/archives/news/webhose/`; filters by `published` date and optionally by `url` domain; enforces 100-word minimum; tags

`source="webhose"`. No HTML fetch needed.

Both write to `rawdata/articles/{shock_id}/` with fields `{id, text, published, url, source, archive_id}`. Test `3DLNewsLoader` on the Kavanaugh 2018 shock, filtering to Sept–Oct 2018 articles from Pennsylvania, Georgia, and Texas local outlets as a demographic cross-section. [2 hrs]

Day 2 (Tue — Social Media Collectors & Bio Keyword Lexicon)

□ **Task:** Implement `electoral/nlp/social.py` with a `SocialCollector` abstract base class and the following concrete subclasses. All write identically-formatted `posts.jsonl`; the scorer is blind to source.

- **BlueSkyCollector:** primary free live path. AT Protocol `app.bsky.feed.searchPosts` via `atproto`. Returns author `description` and `language` fields for bio classification. No rate limit on public posts.
- **ApifyCollector:** secondary free live path. `apify-client` Python SDK running the X Scraper actor. Covers the Evangelical, Catholic, and urban demographic profile that Bluesky underrepresents. Free tier: 500 results/run. **Do not implement XCollector or install tweepy — the X Basic API is not used.**
- **TruthSocialCollector:** Mastodon-compatible `/api/v1/timelines/public?q=` endpoint. No auth. Small user base; weight accordingly.
- **FacebookCollector:** Meta Content Library client if approved, else public page reaction-count fallback.

For the Ayatollah pilot: collect at least 1,000 posts from Bluesky and run Apify for the same event to confirm schema parity. [3 hrs]

□ **Task:** Implement `electoral/nlp/archive.py`: `HistoricalArchiveLoader` class with `load(shock_id, archive_id, window.hours=72, bot.blocklist=None)`. Core behaviour: (i) reads raw archive files from `data/archives/{archive_id}/`; (ii) filters to the shock date window; (iii) drops posts whose `user.id` is in `bot.blocklist` and logs the count; (iv) normalises to `{id, text, created_at, lang, bloc_weights, source, archive_id}`; (v) calls `/classify_batch` for posts with `user.description`; (vi) applies platform proxy for posts without a bio field; (vii) saves to `rawdata/social/archive/{shock_id}/`.

Implement two additional loaders as subclasses or standalone functions:

- **TelegramLoader:** normalises Telegram message objects (field names differ from Twitter — `message` instead of `text`, `date` instead of `created_at`, no `user.description`). Applies Evangelical/conservative Protestant as the platform-level proxy for all posts. Tags with `platform="telegram"`.
- **TikTokLoader:** normalises TikTok video metadata (caption text as `text`, `createTime` as `created_at`, no bio field). Applies younger/secular/diverse as the platform-level proxy. Tags with `platform="tiktok"`.

Handle Twitter schema differences: Kavanaugh uses `full_text` for tweets over 140 characters; normalise all to `text`. Test the main loader on Kavanaugh filtered to Oct 6, 2018 ± 14 days. Test `TelegramLoader` on the USC 2024 dataset. Verify all outputs match the `posts.jsonl` schema exactly. [4 hrs]

□ **Task:** Build the keyword lexicon in `electoral/nlp/bio_classifier.py`: a `BIO_LEXICON` dict mapping each canonical `BlocId` to a list of bio keyword patterns. Source the patterns from Prof. Espinosa’s published research on linguistic and cultural markers per bloc. Required entries per bloc:

- `african_american`: “Black”, “African American”, “HBCU”, “#BlackTwitter”, “NAACP”
- `latino`: “Latino”, “Latina”, “Latinx”, “Chicano”, “Hispanic”, bio in Spanish
- `asian`: “Asian American”, “AAPI”, bio in Chinese/Korean/ Vietnamese/Tagalog/Hindi
- `secular`: “atheist”, “agnostic”, “humanist”, “no religion”, “nonreligious”
- `jewish`: “Jewish”, “#JewishTwitter”, “synagogue”, “rabbi”, bio in Hebrew
- `muslim`: “Muslim”, “Islam”, “Alhamdulillah”, “Inshallah”, “mosque”, bio in Arabic
- `catholic`: “Catholic”, “parish”, “rosary”, “Vatican”, “#CatholicTwitter”
- `protestant`: “Protestant”, “Lutheran”, “Methodist”, “Presbyterian”, “Baptist” (non-Evangelical)
- `evangelical`: “Evangelical”, “born again”, “Southern Baptist”, “#ChristianTwitter”, “Bible-believing”

The bio classifier outputs **three separate probability distributions** per post (one per stratum): `race_weights` over `{african_american, latino, asian, white, other_race}`, `religion_weights` over `{evangelical, catholic, protestant, secular, jewish, muslim, other_rel}`, and `gender_weights` over `{women, men, other_gender}`. These are three independent distributions; no cross-tabulation is produced.

Gender signal (`gender_signal`: "F" | "M" | null) is extracted as a third field from pronouns (“she/her”, “he/him”), explicit words (“woman/women”, “man/men”, “mom”, “feminist”), and name-based inference as a last resort.

Store as: `configs/race_lexicon.json`, `configs/religion_lexicon.json`, `configs/gender_lexicon.json`, all loaded at runtime. [3 hrs]

□ **Task: Language fallback — validation-only, exclude from all estimation.** When both keyword matching and SetFit return no signal (bio empty or below confidence threshold), fall back to language-based Pew priors (e.g. “es” $\rightarrow$ {latino/catholic: 0.38, latino/protestant: 0.22, ...}). Tag every post assigned via this fallback with `inference_method`: “language_prior” in the post record.

Critical: do *not* include language-prior posts in mean μ or covariance Σ_{Δ} estimation. Including them in mean estimation while excluding from covariance bifurcates the data

generating process — μ and Σ would come from different corpora, violating the mean-variance optimizer’s assumptions. Use language-prior posts only as a held-out validation set: after estimation, compare their sentiment signal against the bio-classified signal and report any divergence in the paper. Store language priors in `configs/language_priors.json`. [2 hrs]

□ **Task:** Implement `TruthSocialCollector` and `FacebookCollector` stubs in `social.py`, and `TelegramLoader` and `TikTokLoader` in `archive.py`. Truth Social, Facebook, Telegram, and TikTok posts do not provide structured author bio fields — for all of these apply the platform-level demographic proxy directly. [3 hrs]

Day 3 (Wed — Bio Classifier: Embedding Model, Language Signal & Scorer)

□ **Task:** Build the manual bio labelling dataset: create `data/bio_labels/labeled.bios.jsonl` with 100–200 real user bios per bloc (aim for 1,000+ total). Source bios from the polarization corpus (Kiran et al., already downloaded in Day 1 — ideologically-sorted user bios with labels), the Kaggle political tweets datasets, and the SMAPP NYU elections dataset. For any blocs underrepresented in the archives, manually write example bios from public Bluesky profiles. Bios matching the keyword lexicon from Day 2 can be auto-labelled; focus manual effort on ambiguous cases. [2 hrs]

□ **Task:** Train the **SetFit** bio classifier (Option B): install `setfit`. Load `all-MiniLM-L6-v2` as the embedding backbone. Run `SetFitTrainer(model, train_dataset, eval_dataset)` with the labelled bios; target > 0.75 macro-F1 across all 10 blocs. Export the trained head weights.

Update `scripts/pi_bio_server.py` to handle both execution modes explicitly. On startup, read `configs/base.json["pi_npu_enabled"]`:

- **If true:** load the embedding backbone from the compiled HEF via the Hailo SDK. SetFit classification head runs on CPU as a second step.
- **If false:** load `all-MiniLM-L6-v2` via HuggingFace `SentenceTransformer` into CPU memory. Do not attempt to initialise the Hailo SDK or load any HEF. The SetFit classification head runs on CPU as normal.

Never attempt to pass data to the Hailo interface when NPU is disabled — this causes an immediate crash. The `/health` endpoint must return a JSON object with `"status": "ok"` and `"mode"` set to either `"npu"` or `"cpu"`, so callers can confirm which path is active. Test both modes explicitly before Week 4 Day 3 begins. [3 hrs]

□ **Task:** Implement `electoral/nlp/scorer.py` with **embedding caching** (Option H): `score_text(text)` first checks `data/embeddings/{post_id}.parquet` for a cached embedding vector. If found, skip the model forward pass and use the cached vector directly. If not found, run the model, save the embedding to Parquet, then score. Implement `score_posts_weighted(posts)` using the SetFit bio weights. The first run of the scorer is slow (embeds everything); all subsequent runs with different aggregation logic are near-instant.

Log cache hit rate.

[3 hrs]

Day 4 (Thu — Lagged Favorability Correlation & Elasticity Regression)

□ **Task:** Generate **synthetic training data** using Gemini 2.5 Pro (Option D). Run `scripts/generate_synthetic.py`: (i) construct a single prompt containing the full voter panel (as CSV), all 25+ real shock records, the canonical stratum IDs, and instructions to generate 500–1,000 plausible hypothetical shock scenarios with demographically grounded delta estimates per stratum; (ii) parse the JSON response into JSONL records matching the fine-tuning schema; (iii) mark each record `"synthetic": true` and assign an `mmd.weight` per the formula below; (iv) save to `data/finetune/synthetic.jsonl` and diagnostics to `data/finetune/synthetic_diagnostics.json`.

Three required diagnostics (compute for every batch; store in diagnostics file):

- **MMD:** compute RBF-kernel MMD between synthetic and real joint $\Delta\mu$ distributions. Set $\lambda = 1/(2\sigma^2)$ where $\sigma^2 = \text{median pairwise distance over the real data (median heuristic)}$. Assign weight $w = 0.5 \times \exp(-\lambda \cdot \text{MMD}^2)$.
- **PCA alignment:** fit PCA on real $\Delta\mu$ matrix; project synthetic batch onto the top-2 PCs; flag if $< 70\%$ of synthetic variance is explained.
- **PCD (Pairwise Correlation Difference):** compute Frobenius norm of (real pairwise correlation matrix – synthetic pairwise correlation matrix) normalized by number of off-diagonal elements. Log for paper.

Never use a hard KS-test rejection gate — the soft MMD weight is sufficient.

[3 hrs]

□ **Task:** Build `configs/market_contracts.json`: a static mapping of every shock event to its exact contract identifier on each supported prediction market platform. Querying Polymarket, Manifold, or Metaculus with a generic slug like `kavanaugh.2018` returns empty or irrelevant results — each platform uses its own internal IDs that must be looked up manually. Each entry maps a shock slug to a dict of `{platform: contract_id_or_null}`, for example: `"kavanaugh.2018": {"polymarket": null, "predictit": "6274", "metaculus": "3712", "manifold": null, "iem": "PreVote18"}`. For each of the 25+ shock events, visit each platform's search interface, find the closest relevant contract, and record the exact ID. Mark `null` where no contract exists. This file is the source of truth for the market collector — no dynamic slug resolution. Commit to the repo and add to the replication package.

[2 hrs]

□ **Task:** Implement **prediction market collection** in `electoral/markets/collector.py`. Load contract identifiers from `configs/market_contracts.json` (built in the previous task) — never resolve slugs dynamically; use only the hardcoded IDs. For each of the 25+ shock events, collect the market-implied win probability for both the Democrat and Republican candidate contracts at $t - 24\text{h}$, $t + 1\text{h}$, $t + 24\text{h}$, and $t + 72\text{h}$. API endpoints:

- **Polymarket** (2020+): `clob-api.polymarket.com` REST API. Free, no auth required. Query by CLOB market address from `market_contracts.json`.

- **PredictIt** (2014+): www.predictit.org/api/marketdata/all/ JSON feed. Free, no auth. Query by contract ID from `market_contracts.json`. Download full historical CSV from predictit.org/research.
- **Metaculus** (2015+): www.metaculus.com/api2/questions/ Free REST API. Query by question ID from `market_contracts.json`.
- **Manifold** (2022+): manifold.markets/api/v0/markets Free REST API. Query by market slug from `market_contracts.json`.
- **Iowa Electronic Markets** (1988+): Download historical CSV from tippie.uiowa.edu/iem. Parse offline. Best coverage for 2000–2014.

Implement `electoral/markets/aggregator.py`: `volume_weighted_price(market_prices, volumes)` that weights each market's price by its trading volume; assign equal weight when volume is unavailable (Metaculus, Manifold). Construct a `PredictionMarketData` artifact per shock event and write to `artifacts/markets/{shock_id}.json`. For events before 2014, set `prediction_market = null` in the fine-tuning record. [3 hrs]

□ **Task:** Implement `electoral/nlp/elasticity.py`: `fit_elasticity(news_scores, social_scores, polling_deltas)` that regresses observed vote-share deltas on both news RoBERTa scores and baseline-adjusted social media scores, using platform as a demographic proxy weight. Use `sklearn.linear_model.Ridge` with cross-validation on held-out shock events. Store coefficients per source (news vs. each platform vs. archive) per bloc. For shocks with archive coverage, the archive posts contribute to the same `social_scores` dict as live posts — they are already normalised to the same schema and the scorer has processed them identically. [3 hrs]

□ **Task:** Compute and log the per-platform correlation between baseline-adjusted social sentiment and 14-day lagged favorability deltas. Write results to `docs/platform_correlations.md`. If a platform shows near-zero correlation, down-weight or exclude it in the regression — document the decision. [2 hrs]

Day 5 (Fri — Merge Stage, Dataset Assembly, Full Corpus Run & PR)

□ **Task:** Implement `merge_posts(shock_id)` in `electoral/kernels/sentiment.py` as a Prefect task. This stage runs *before* the RoBERTa scorer and is required because multiple files may exist per shock (one per platform per collector run). The task: (i) finds all `rawdata/social/{platform}/{shock_id}.jsonl` files across all platforms and all archive sources; (ii) concatenates them into a single `rawdata/merged/{shock_id}/posts.jsonl`; (iii) logs the total post count broken down by platform so you can see the coverage split. The scorer always reads from `rawdata/merged/` and is completely blind to how many files or platforms contributed. Wire this task into `pipeline.py` as the stage immediately preceding `score_posts`. Verify: running the pipeline twice produces identical merged files. [2 hrs]

□ **Task:** Implement `build_finetune_dataset` in `elasticity.py`: create one JSONL record

per shock event using the unified schema. The `social_roberta_scores` field is keyed by `bloc_id`; include the `prediction_market` block from the `PredictionMarketData` artifact (set to `null` for pre-2014 events). Apply differential weighting: market-backed examples get `weight=1.5`, poll-only examples get `weight=1.0`, synthetic examples get `weight=0.5`. Write to `data/finetune/train.jsonl` (all cycles except 2020) and `data/finetune/eval.jsonl` (2020 only). Validate: no duplicate shock IDs, all 10 bloc keys present, no train/eval overlap. [3 hrs]

□ **Task:** Submit the RoBERTa scoring pipeline as a **HPC SLURM array job** across all 25+ shock events simultaneously.

First: confirm sampled and cleaned data is in place. The scorer reads from `rawdata/merged/{shock_id}/posts.jsonl` which should contain the output of `sample_archives.py` and `clean_with_llm.py`, not the raw full-corpus files. Confirm post counts are in the 5,000–200,000 range per shock, not millions. If raw data was accidentally staged, re-run sampling first.

Then: transfer and submit. Do not rely on Syncthing on HPC nodes — scratch nodes kill background daemons. Run:

- `rsync -avz rawdata/merged/ hpc:$SCRATCH/electoral/merged/`
- `rsync -avz rawdata/articles/ hpc:$SCRATCH/electoral/articles/`

Wait for `rsync` to complete before submitting the job.

Then: write and submit scripts/score_array.sh: `#SBATCH --array=0-24`, one node per shock event, each node loads its assigned `posts.jsonl` from `$SCRATCH/electoral/merged/` and runs the weighted scorer. Collecting 25 events serially on the M5 would take 2–3 days; the array job finishes in under 2 hours on the HPC. Monitor with `squeue -u $USER`. [3 hrs]

□ **Task:** Write `tests/test_bio_classifier.py`: (i) keyword match for an explicit Evangelical bio returns `evangelical` weight > 0.8 , (ii) Spanish-language `lang` code boosts `latino` weight, (iii) `combine_signals` output sums to 1.0 for all inputs, (iv) unclassifiable bio returns platform fallback. Write `tests/test_social.py`: (i) `BlueSkyCollector` records include `author.description` and `lang` fields, (ii) `ApifyCollector` output schema matches `BlueSkyCollector` exactly, (iii) baseline adjustment produces per-bloc scores in $[-1, 1]$, (iv) weighted scorer produces 10 distinct elasticity values not a single platform score. Write `tests/test_sentiment.py`: (i) scorer outputs sum to 1.0, (ii) dataset has no train/eval overlap. [1 hrs]

□ **Task:** Weekly PR: `feature/week4-roberta-social-pipeline`. Include: dataset statistics (examples, shock categories, mean elasticity per bloc), platform correlation table, and the Ayatollah pilot findings. [1 hrs]

□ **Task:** Brief supervision check-in with Prof. Espinosa: share platform correlation results and elasticity scores for 3–4 key shocks. Get domain input on whether Truth Social’s Evangelical reactions align with his published patterns. Juhi decides on platform weighting adjustments

and documents in DECISIONS.md.

[2 hrs]

5 Week 4 — LLM Fine-Tuning & Inference Endpoint (40 hrs)

Goal. By Friday, Mistral 7B is fine-tuned via QLoRA on the unified news + social media dataset on HPC, the FastAPI endpoint returns valid `ShockResponseData` JSON with constrained decoding, and the three-way baseline comparison (news-only, social-only, unified) is documented.

End condition.

- ☐ 100 consecutive inference calls all return schema-valid JSON.
- ☐ Endpoint responds in under 3 seconds on M5.
- ☐ Three-way MAE comparison (news-only, social-only, unified) committed.
- ☐ PR merged.

Day 1 (Mon — HPC Setup & Local MLX Baseline)

☐ **Task:** Set up CMC HPC access: log in via SSH, confirm SLURM quota and available GPU nodes (`sinfo`), check scratch storage allocation (`quota`). Create the project directory at `$SCRATCH/electoral-equilibrium/`. Document node names, partition names, and max walltime in `scripts/HPC_NOTES.md`. [1 hrs]

☐ **Task:** Copy the fine-tuning dataset to HPC scratch: `scp -r data/finetune/hpc:$SCRATCH/electoral-equilibrium/data/`. Verify file counts match locally. Set up a Python virtual environment on HPC: `python -m venv $SCRATCH/venv` and install `transformers`, `peft`, `bitsandbytes`, `torch`, `accelerate`. Confirm `import torch; torch.cuda.is_available()` returns `True` on a GPU node. [2 hrs]

☐ **Task:** Write `scripts/hpc_submit.sh`: a SLURM batch script that requests 1 GPU (A100 if available, else V100), 32GB RAM, 4 CPU cores, and 4-hour walltime. The script activates the venv, sets `TRANSFORMERS_CACHE` to scratch, and calls `python -m electoral.llm.trainer --backend hpc --config configs/train_r16.json`. Test with `sbatch --test-only scripts/hpc_submit.sh` to validate the script parses without errors. [2 hrs]

☐ **Task:** Install MLX and `mlx-lm` locally on the M5: `pip install mlx mlx-lm`. Download `mistralai/Mistral-7B-v0.3` weights in MLX format: `mlx_lm.convert --hf-path mistralai/Mistral-7B-v0.3 --mlx-path models/mistral-7b-mlx`. Confirm the model loads and generates a test completion. This download is ~14GB — start it immediately and work on other tasks while it runs. [1 hrs]

☐ **Task:** Run a 10-step smoke fine-tune locally with MLX: `mlx_lm.lora --model models/mistral-7b-mlx --train --data data/finetune/train.jsonl --batch-size 2 --lora-layers 4 --iters 10`. Confirm the run completes, loss decreases over 10 steps, and an adapter file is saved. Log the output to `docs/mlx.smoke_run.txt`. [1 hrs]

□ **Task:** Implement `electoral/llm/trainer.py`: `TrainConfig` dataclass with fields: `base_model`, `lora_rank`, `lora_alpha`, `learning_rate`, `epochs`, `batch_size`, `train_path`, `eval_path`, `output_dir`, `backend` (“mlx” or “hpc”). Write `load_config(path)` and `save_config(config, path)`. Create `configs/train_r16.json` and `configs/train_r32.json` with the hyperparameters from the development plan. [1 hrs]

Day 2 (Tue — Prompt Engineering & First Training Run)

□ **Task:** Design the prompt template. Write three candidate templates in `docs/prompt_candidates.md` and test each on 5 training examples manually using the base (un-fine-tuned) Mistral 7B locally. Evaluate: does the model output valid JSON? Does the structure make the task obvious? Select the best template and document the reasoning. [2 hrs]

□ **Task:** Implement the selected prompt template in `trainer.py`: `format_prompt(example)` that takes a JSONL record and returns a formatted string. The prompt must include: (i) system context paragraph establishing the political science framing and the selected party; (ii) the event description; (iii) RoBERTa news scores as a labeled JSON block (per-bloc); (iv) X bio-weighted social scores as a labeled JSON block (per-bloc, sourced from `social_roberta_scores`); (v) the historical analog and shock category; (vi) a closing instruction: “You are analyzing the impact on the [party] candidate’s coalition. Output only a JSON object with three keys: `delta_bins_race` (keys: african_american, latino, asian, white, other_race), `delta_bins_religion` (keys: evangelical, catholic, protestant, secular, jewish, muslim, other_rel), and `delta_bins_gender` (keys: women, men, other_gender). Values are one of: strong_neg, mod_neg, mild_neg, slight_neg, neutral, slight_pos, mild_pos, mod_pos, strong_pos. Also output `delta_eff` (scalar): the pipeline will verify it matches $\lambda_1 \sum w_i \Delta \mu_i^{\text{race}} + \lambda_2 \sum v_R \Delta \mu_R^{\text{rel}} + \lambda_3 \sum g_G \Delta \mu_G^{\text{gen}}$. For non-gendered shocks, gender bins may be identical across women/men.” The party name must be substituted from `example["party"]`, never hardcoded. [2 hrs]

□ **Task:** Implement the HPC training path in `trainer.py`: when `backend='hpc'`, use HuggingFace `transformers` + `peft.LoraConfig` + `BitsAndBytesConfig` (4-bit NF4). Implement `train_hpc(config)` that loads the base model in 4-bit, applies LoRA adapters to Q/K/V/O projections, runs `Trainer.train()`, and saves the adapter to `config.output_dir`. [2 hrs]

□ **Task:** Submit the first full HPC training run: `sbatch scripts/hpc-submit.sh` with `configs/train_r16.json`. Monitor the job queue with `squeue -u $USER`. Once the job starts, `tail -f` the SLURM output file to watch the loss curve. The job should complete overnight. If it times out, increase walltime and resubmit. [1 hrs]

□ **Task:** While the HPC job runs: implement `electoral/llm/eval.py`. `EvalResult` dataclass: `shock_id`, `mae_per_bloc` (dict), `mean_mae`, `output_json` (raw model output). `evaluate_model(adapter_path, eval_path)` loads the adapter, iterates through each eval

example, runs `estimate_shock`, computes MAE against ground-truth deltas, and returns a list of `EvalResult`. Write `print_eval_summary(results)` that logs a table of MAE per bloc sorted descending. [1 hrs]

Day 3 (Wed — Constrained Decoding & Inference Wrapper)

□ **Task:** Install `outlines`: `pip install outlines`. Confirm it imports correctly alongside the existing HuggingFace stack. Read the `outlines.generate.json` documentation. Write a standalone 10-line proof-of-concept script `scripts/outlines_poc.py` that loads a tiny HuggingFace model (`gpt2`), generates constrained JSON with a simple schema (`{'x': int}`), and prints the result. Confirm the output is always valid JSON regardless of how many times you run it. [1 hrs]

□ **Task:** Convert `ShockResponseData` to a **Pydantic** model in `electoral/artifacts.py` (or a parallel `ShockResponseSchema` Pydantic class if keeping the frozen dataclass for internal use). Define the 10 bloc fields as `Annotated[str, Literal["strong_neg", "mod_neg", "mild_neg", "weak_neg", "neutral", "weak_pos", "mild_pos", "mod_pos", "strong_pos"]]` — one field per canonical bloc. Passing this Pydantic model directly to `outlines.generate.json(model, ShockResponseSchema)` has two critical advantages over a manual `json_schema()` dict: (i) the LLM output schema and the FastAPI `response_model` are the *same Python object*, so they can never drift out of sync; (ii) Pydantic's `Literal` type maps cleanly to the 9-token enum constraint that constrained decoding enforces. Write a unit test confirming the Pydantic model validates a correct output and rejects an unknown bin token. [1 hrs]

□ **Task:** Implement `electoral/llm/inference.py`. Class `ShockEstimator` with `__init__(self, adapter_path, base_model)` that loads the model once and keeps it in memory. Method `estimate(self, event, intensity)` that: (i) formats the prompt via `format_prompt`; (ii) passes the `ShockResponseSchema` Pydantic model directly to `outlines.generate.json(model, ShockResponseSchema)` — no manual dict construction; (iii) runs generation; (iv) maps bin tokens to numeric deltas via `bin_to_delta`; (v) scales delta values by `intensity`; (vi) constructs and validates `ShockResponseData`; (vii) returns it. The `ShockResponseSchema` Pydantic model is also used as the FastAPI `response_model` in `shock_endpoint.py`, guaranteeing the two schemas are always identical. [3 hrs]

□ **Task:** Write `tests/test_constrained_output.py`: instantiate `ShockEstimator` with the trained adapter. Run `estimator.estimate(event, intensity)` 100 times with 10 different event strings (10 calls each). Assert: (i) every output passes `ShockResponseData.validate()`, (ii) all delta values are finite and in $[-0.15, 0.15]$, (iii) all 10 bloc keys are present, (iv) no call raises an exception. [2 hrs]

□ **Task:** Benchmark latency: time 20 sequential calls to `estimator.estimate()` with a standard Ayatollah event description. Print mean, median, and p95 latency. If p95 exceeds 3 seconds: (i) confirm the model is loaded once (not reloaded per call), (ii) try reducing

`max_tokens` in the generator, (iii) try 4-bit quantization if not already active. Document the final p95 in `docs/latency_benchmark.md`. [1 hrs]

Day 4 (Thu — FastAPI Endpoint & Baseline Comparison)

□ **Task:** Implement `electoral/api/shock_endpoint.py`. Use FastAPI lifespan context to load `ShockEstimator` once at startup and store in `app.state.estimator`. Define three routes: `POST /estimate` (body: `EstimateRequest(event: str, intensity: float = 1.0)`, returns `ShockResponseData`), `GET /health` (returns `{"status": "ok", "model": adapter_path}`), `GET /blocs` (returns list of canonical bloc names from `types.py`). Add `CORSMiddleware` allowing `http://localhost:3000` for local development. [2 hrs]

□ **Task:** Add request validation and error handling to the endpoint. If `event` is empty or under 10 characters, return HTTP 422 with message “Event description too short”. If `intensity` is outside `[0.1, 3.0]`, return HTTP 422 with message “Intensity out of range”. Wrap the `estimator.estimate()` call in a try/except: on any unexpected error, return HTTP 500 with a sanitized message (no stack trace in the response body, but log the full trace server-side). [2 hrs]

□ **Task:** Run the three-way baseline comparison on the 2020 eval set. For each method, call the appropriate function on every eval example and collect `EvalResult` lists: (i) news-only: use the `SentimentData` artifact’s news RoBERTa scores mapped through the elasticity regression; (ii) social-only: same but using only platform scores; (iii) unified LLM: use `estimator.estimate()`. Compute mean MAE per bloc and overall. Save to `artifacts/baseline_comparison.json` and `docs/baseline_comparison.md` with a formatted table. [2 hrs]

□ **Task:** Write `tests/test_endpoint_latency.py`: start the FastAPI app as a subprocess with `subprocess.Popen`, wait for the health check to return 200, then send 5 POST requests to `/estimate` with the Ayatollah event description. Assert each response: (i) HTTP 200, (ii) valid JSON, (iii) parses into `ShockResponseData` without error, (iv) response time under 3 seconds. Teardown: terminate the subprocess. [2 hrs]

Day 5 (Fri — Hyperparameter Decision, Kernel Wiring & PR)

□ **Task:** Compare the $r = 16$ and $r = 32$ eval results. Decision rule: adopt $r = 32$ if mean MAE improves by more than 5% across all blocs. If neither run is satisfactory (mean MAE > 0.05 vote-share points), consider the Qwen2.5 14B fallback: re-run the smoke MLX test with `Qwen/Qwen2.5-14B-Instruct` and compare structured output reliability. Document the final model and rank decision in `DECISIONS.md` with the supporting MAE numbers. [2 hrs]

□ **Task:** Retrieve the winning adapter weights from HPC scratch using `scp`, not `SyncThing`. HPC scratch nodes frequently kill background daemons including `SyncThing`;

relying on it for model weight transfer risks silent failures or corrupted weights from race conditions. Run: `scp -r hpc:$SCRATCH/electoral-equilibrium/adapters/mistral-7b-electoral/ adapters/`. Verify the adapter loads correctly on M5. Tag the version in `adapters/mistral-7b-electoral/VERSION` with the HPC job ID, training config path, and eval MAE. After verifying the transfer, add the adapter directory to Syncthing for local machine availability. [1 hrs]

□ **Task:** Wire `electoral/kernels/finetune.py`: implement `build_llm_finetune(config)` that reads `LLMFineTuneData` from the artifact store (or constructs it from the adapter directory if already trained). Wire into `stages.py`. The finetune stage is idempotent: if an adapter already exists at the configured path, skip training and log “adapter found, skipping training”. [2 hrs]

□ **Task:** Wire `electoral/kernels/shock.py`: implement `build_shock_response(config, event, intensity)` that calls `estimate_shock`, `bootstrap_cov`, and `solve_rebalanced` in sequence, wraps results in `ShockResponseData` and `EquilibriumData` payloads, and writes both artifacts. Run the full pipeline smoke config end-to-end including the shock stage. [2 hrs]

□ **Task:** Weekly PR: `feature/week5-llm-finetune`. PR description must include: (i) the three-way MAE comparison table, (ii) the model and rank decision with justification, (iii) HPC job IDs and adapter path, (iv) a sample of 3 eval examples showing input event, predicted deltas, and ground-truth deltas side by side. [1 hrs]

□ **Task:** Brief supervision check-in with Prof. Espinosa: share 3–4 eval examples where the model’s predictions diverge most from ground truth. Get domain perspective on whether divergences reflect real political dynamics. Juhi decides on any model adjustments. [2 hrs]

6 Week 5 — Stochastic Optimization & Monte Carlo (40 hrs)

Goal. By Friday, the CVXPY optimizer rebalances voter distributions after a shock, 10,000+ Monte Carlo simulations run on HPC, and the `SimulationData` artifact is produced and validated.

End condition.

- ☐ Optimizer produces feasible weights for all test shocks.
- ☐ Monte Carlo win probability converges within ± 0.005 between $N = 5,000$ and $N = 10,000$.
- ☐ All optimization and simulation tests pass.
- ☐ PR merged.

Day 1 (Mon — Post-Shock Optimization)

☐ **Task:** Implement `electoral/optimization/simplex.py: project_simplex(v)` using the Duchi et al. (2008) $O(n \log n)$ algorithm. The algorithm: sort v descending, find the largest ρ such that $v_{(\rho)} - \frac{1}{\rho}(\sum_{j \leq \rho} v_{(j)} - 1) > 0$, set $\theta = \frac{1}{\rho}(\sum_{j \leq \rho} v_{(j)} - 1)$, return $\max(v - \theta, 0)$. Write three unit tests: (i) already-valid simplex vector returns unchanged, (ii) negative components are zeroed and remainder redistributed, (iii) output always sums to 1.0 within `tol=1e-9`. [3 hrs]

☐ **Task:** Implement `electoral/optimization/cvx.py: solve_rebalanced(mu_tilde, cov_delta, target)` using CVXPY with the **max P(win) objective** (not min-variance). Formulate the Sharpe-ratio objective: `numerator = mu_tilde @ w - target` (linear in w); `denominator = cp.sqrt(cp.quad_form(w, cov_delta))` (convex in w); `objective = cp.Maximize(cp.ratio(numerator, denominator))`. Declare `problem.solve(qcp=True)` (Disciplined Quasiconvex Programming). **Assert** `problem.is_dcp(qcp=True) == True` **before solving** — if this assertion fails, the objective is not quasi-convex and the solver cannot guarantee the global optimum. This assertion must be a unit test. On `SolverError` or infeasibility, return `EquilibriumData` with `feasible=False`. [4 hrs]

☐ **Task:** Implement `solve_equal_weight_rebalanced(blocs, delta_w)` as a fallback in `cvx.py`: if the QP is infeasible after 5 retries, return an equal-weight distribution with `feasible=False` and `target_met=False`. Log a warning. This ensures the web app never crashes due to an infeasible shock input. [2 hrs]

Day 2 (Tue — Covariance Bootstrap & Full Shock Chain)

☐ **Task:** Implement `electoral/models/bootstrap.py` using **Ledoit-Wolf shrinkage** rather than raw bootstrap covariance. With only 6–8 election cycles, bootstrapping a 10×10 covariance matrix yields a rank-deficient result (≤ 7 degrees of freedom for 55 parameters). Applying `psd_repair` to a rank-deficient matrix just adds noise — the optimizer would be minimizing against your artificial ϵI injection rather than historical electoral reality.

Use `sklearn.covariance.LedoitWolf` instead: it applies an analytically optimal shrinkage toward a diagonal target, is specifically designed for the $p > n$ regime, and produces a well-conditioned, full-rank covariance estimate. Usage: instantiate `LedoitWolf(assume_centered=False)`, call `.fit(delta_matrix)` where `delta_matrix` has shape `(n_cycles, 10)`, read `.covariance_` — always full-rank, no `psd_repair` needed. Keep `bootstrap_cov` as an alternative for comparison; document in `DECISIONS.md` that Ledoit-Wolf is the primary estimator. Write unit tests: (i) output is symmetric and PSD, (ii) condition number is finite and below 10^6 , (iii) same input produces identical output. [3 hrs]

□ **Task:** Implement the sensitivity-weighted covariance variant `bootstrap_cov_weighted(panel_df, social_elasticities, n_bootstrap=1000, seed=42)`: weight each cycle's contribution by its mean absolute social media elasticity across platforms, so cycles with stronger social media reactions contribute more to the covariance estimate. This fuses the social media signal directly into the stochastic model. Store both variants; the paper will compare them. [2 hrs]

□ **Task:** Integrate the full shock response chain in `electoral/kernels/shock.py`: `build_shock_response(config, event_description, intensity)`. Step 1: call `estimate_shock` to get `delta_mu` dict ($\Delta\mu_i$ per bloc). Step 2: load μ and $w^{(0)}$ from the `BaselinePortfolioData` artifact. Step 3: compute $\tilde{\mu} = \mu + \Delta\mu$. Step 4: call `bootstrap_cov` to get Σ_Δ . Step 5: call `solve_rebalanced(mu_tilde, cov_delta, target)` to produce `EquilibriumData` (which stores both $\tilde{w}$ and $\tilde{\mu}$). Step 6: construct and validate `ShockResponseData` wrapping `delta_mu` and the covariance. Wire into `stages.py`. [3 hrs]

Day 3 (Wed — Monte Carlo Engine)

□ **Task:** Implement `electoral/simulation/montecarlo.py`: `run_simulations(w_star, mu_tilde, cov_delta, target, n, seed)`. Use the **Logistic-Normal with ILR parameterization** — not Dirichlet (which forces negative off-diagonal covariances) and not Gaussian-then-project (which destroys covariance structure asymmetrically at simplex boundaries).

Implementation steps:

- Construct a Helmert contrast matrix $V \in \mathbb{R}^{(K-1) \times K}$.
- Map w^* to ILR coordinates: $z^* = V^\top \log(w^*)$
- Propagate Σ_Δ to ILR space via the Jacobian J of the ILR transformation: $\Sigma_{\text{ILR}} = J \Sigma_\Delta J^\top$
- Draw N samples: $y^{(n)} \sim \mathcal{N}(z^*, \Sigma_{\text{ILR}})$
- Back-transform via softmax: $w_i^{(n)} = \exp(y_i^{(n)}) / \sum_j \exp(y_j^{(n)})$
- Compute win flags and confidence interval bounds (5th and 95th percentiles)

If $w_i^* = 0$ exactly for any bloc, report it as `infeasible_bloc` in `EquilibriumData` and exclude it from the simulation — do *not* floor to 0.01 (flooring distorts log-ratio space non-linearly).

Required test: `test_montecarlo_degenerate` — with $w_0^* = 0.99$, $w_{1..4}^* = 0.0025$, assert all

N samples sum to 1.0 within $\text{tol}=10^{-9}$ and win_probability is in $[0, 1]$. [5 hrs]

□ **Task:** Implement `project_simplex_batch(W)` in `simplex.py`: takes an $(N \times d)$ matrix and returns each row projected onto the probability simplex. This function is a **fallback utility** used only when the CVXPY optimizer returns an infeasible solution and equal-weight distribution is substituted — it is *not* the primary Monte Carlo sampling path (which uses Dirichlet and produces simplex-valid samples directly).

Implement Duchi's $O(n \log n)$ algorithm per row. A Python `for` loop over rows is acceptable and correct here: at $N = 10,000$, $d = 10$, the dominant cost is the sort, and Python overhead per row is negligible. Vectorizing the dynamic per-row threshold ρ in pure NumPy is brittle without a loop — do not attempt it. If the fallback path ever becomes a throughput bottleneck (it won't during the SRP), use `numba.jit` to compile the inner loop.

Tests: (i) all rows sum to 1.0 within $\text{tol}=1\text{e-}9$, (ii) all values non-negative, (iii) already-valid row returns unchanged, (iv) $N = 10,000$ completes in under 5 seconds on M5 (loop is fine). [2 hrs]

□ **Task:** Write `scripts/run_montecarlo_hpc.sh`: SLURM batch script requesting 8 CPU cores, 16GB RAM, 2-hour walltime. The script loads a shock event from `artifacts/shock_{id}.json`, runs `python -m electoral.simulation.montecarlo` with $N = 50,000$, and saves `SimulationData` to `artifacts/sim_{id}.json`. Submit for the Ayatollah event as a test job. Confirm the job completes and produces a valid artifact. [1 hrs]

□ **Task:** Write `tests/test_montecarlo.py`: (i) smoke run $N = 100$ completes in under 2 seconds on M5, (ii) win probability is in $[0, 1]$, (iii) with μ all equal to 0.8 and `target=0.535`, win probability > 0.95 , (iv) with μ all equal to 0.2, win probability < 0.05 , (v) two runs with same seed produce identical `SimulationData.to_dict()` outputs, (vi) percentiles list has exactly 5 values per bloc in $[0, 1]$. [1 hrs]

Day 4 (Thu — Convergence Tests, Sensitivity Analysis & Kernel Wiring)

□ **Task:** Write `tests/test_mc_convergence.py`: run simulations at $N = 1,000$, $N = 5,000$, and $N = 10,000$ with the same seed. Assert the $N = 5,000$ and $N = 10,000$ win probability estimates differ by less than 0.005. Also assert the $N = 1,000$ estimate is within 0.02 of the $N = 10,000$ estimate (looser bound). Log the three estimates side by side. [2 hrs]

□ **Task:** Write `tests/test_optimization.py`: (i) a feasible shock (small delta, weak covariance) returns weights summing to 1.0 with `feasible=True` and `target_met=True`, (ii) an extreme shock that pushes the portfolio below target returns `target_met=False` and logs the relaxation steps, (iii) `project_simplex` output always non-negative and sums to 1.0, (iv) same inputs produce identical `EquilibriumData.to_dict()` across two calls. [2 hrs]

□ **Task:** Implement a shock sensitivity analysis script `scripts/sensitivity_analysis.py`: for a given shock event, vary the intensity parameter from 0.5 to 2.0 in steps of 0.25 and plot how win probability and per-bloc weights change. Save the plot as `artifacts/sensitivity_{shock_id}.png`. Run on the Ayatollah event as a case study. This figure will be useful in the paper. [2 hrs]

□ **Task:** Wire `electoral/kernels/optimize.py` into `stages.py`: `build_optimization(config, shock_response_artifact)` that loads the `ShockResponseData`, runs `solve_rebalanced`, and writes `EquilibriumData`. Run the full pipeline smoke config end-to-end and confirm all seven artifact files exist and validate. [2 hrs]

Day 5 (Fri — Metrics, Reporting & PR)

□ **Task:** Implement `electoral/metrics/performance.py`. Required functions: `win_probability(sim_data)` (reads from `SimulationData`), `equilibrium_gap(weights, mu, target)` returning $\mathbf{w}^\top \boldsymbol{\mu} - V_{\text{eq}}$ (positive = above threshold), `bloc_delta_summary(baseline_weights, rebalanced_weights)` returning a dict of signed deltas per bloc sorted by absolute magnitude. All functions must accept and return JSON-native types. [2 hrs]

□ **Task:** Implement `electoral/metrics/tables.py`: write `build_shock_summary_table` (takes `shock_ids` and `config`) that iterates over shock event IDs, loads the corresponding `EquilibriumData` and `SimulationData` artifacts, and assembles a summary table with columns: shock id, category, win probability, equilibrium gap, top-moving bloc, and top-moving delta. Return as a JSON-serializable list of dicts. [2 hrs]

□ **Task:** Implement `electoral/reporting/export.py`: `export_json(path, data)`, `export_csv(path, rows, fieldnames)`, and `export_latex_table(path, rows, caption, label)` that produces a booktabs-style LaTeX table fragment. Run the summary table for all 25+ shocks and save to `artifacts/shock_summary_table.csv` and `artifacts/shock_summary_table.tex`. [2 hrs]

□ **Task:** Weekly PR: `feature/week6-optimization-montecarlo`. Include the convergence test output, the sensitivity analysis plot for the Ayatollah event, and the shock summary table (first 10 rows) in the PR description. [1 hrs]

□ **Task:** Brief supervision check-in with Prof. Espinosa: share win probability estimates for 4–5 historical shocks. Get domain input on whether results align with known electoral outcomes. Juhi decides which covariance variant to use in the paper and documents the decision. [1 hrs]

7 Week 6 — Web Application (40 hrs)

Goal. By Friday, the core public web application is live end-to-end with SSE streaming and deployed on Modal + Vercel. The analyst dashboard moves to Week 8 Days 1–2 to give it the time it actually requires.

End condition.

- ☐ Public app renders per-bloc shifts progressively via SSE.
- ☐ Audit log records every `/estimate` call with latency.
- ☐ Modal + Vercel deployment live with public URL.
- ☐ PR merged.

Day 1 (Mon — Next.js Setup & Typed API Client)

☐ **Task:** Initialize the Next.js app: `npx create-next-app webapp --typescript --tailwind --app --no-src-dir`. Remove all boilerplate page content. Set up `webapp/.env.local`: `NEXT_PUBLIC_API_URL= http://localhost:8000`. Confirm `npm run dev` starts without errors and the browser shows a blank page. [1 hrs]

☐ **Task:** Install frontend dependencies: `npm install recharts zod @radix-ui/react-slider @radix-ui/react-collapsible lucide-react`. These cover charting, schema validation, the intensity slider, the “How it works” expandable, and icons respectively. [1 hrs]

☐ **Task:** Define TypeScript types in `webapp/lib/types.ts`: all existing payload types (`ShockResponseData`, `EquilibriumData`, `SimulationData`, `EstimateResponse`) plus new dashboard types: `AuditEntry` (`id`, `timestamp`, `event_text`, `party`, `intensity`, `win_prob`, `feasible`, `llm_ms`, `optimizer_ms`, `montecarlo_ms`), `CoverageCell` (`bloc_id`, `cycle`, `quality`, `vote_share`, `sources`), and `TrainingRun` (`run_id`, `epochs`, `train_loss`, `val_loss`, `mae`). Add `Party = "democrat" | "republican"` as a shared literal type. All types must mirror their FastAPI response models exactly. [1 hrs]

☐ **Task:** Implement Zod schemas in `webapp/lib/schemas.ts` for each type. These are used to validate API responses at runtime so the frontend fails loudly rather than rendering NaN or undefined. Export `EstimateResponseSchema.parse(data)` as the single validation entry point. [1 hrs]

☐ **Task:** Implement `webapp/lib/api.ts`: `estimateShock(event: string, intensity: number): Promise<EstimateResponse>` that POSTs to `/estimate`, parses the response with the Zod schema, and throws a typed `ApiError` (with `message` and `status`) on any failure. Implement `getBlocs(): Promise<string[]>` and `healthCheck(): Promise<boolean>`. [2 hrs]

□ **Task:** Write a smoke integration test for the API client: in `webapp/lib/__tests__/api.test.ts`, mock `fetch` using `jest.fn()`, provide a valid `EstimateResponse` fixture, and assert: (i) the correct URL is called, (ii) the response parses without error, (iii) a malformed response throws `ApiError`. [2 hrs]

Day 2 (Tue — ShockInput Component & State Management)

□ **Task:** Implement the page-level state in `webapp/app/page.tsx` using `useState`: `party` ("democrat" | "republican", default "democrat"), `event` (string), `intensity` (number, default 1.0), `loading` (boolean), `error` (string — null), `deltaBins` (`Record<string, string>` | null), `shifted` (`Record<string, number>` | null), `equilibrium` (`EquilibriumData` | null), `simulation` (`SimulationData` | null).

Implement `handleSubmit`: open an SSE `EventSource` to `/estimate/stream`; on `event: deltas` set `deltaBins` and `shifted` (populates `ShockNarrative` immediately, and renders `CoalitionChart` with opaque bars only — rebalance layer is absent until the next event); on `event: equilibrium` set `equilibrium` (adds the translucent rebalance layer to `CoalitionChart` in-place — no re-render of the opaque bars); on `event: simulation` set `simulation` (populates `WinGauge`); on `[DONE]` set `loading=false`.

Layout: `ShockNarrative` above the chart, full width. `CoalitionChart` below it, full width (the chart is self-contained with both layers). `WinGauge` to the right on desktop, below on mobile. The progressive reveal — opaque bars first, translucent overlay second, gauge last — gives the user something to read at each SSE step. [3 hrs]

□ **Task:** Implement `webapp/components/ShockInput.tsx`. Props: `party`, `setParty`, `event`, `setEvent`, `intensity`, `setIntensity`, `onSubmit`, `loading`. Render: (i) a two-button toggle labeled **Democrat** / **Republican**, styled blue/red respectively, that sets `party` state — this is the first control the user sees; (ii) a `<textarea>` with placeholder “Describe a hypothetical political event...”, minimum 3 rows, full width; (iii) a labeled `Slider` (Radix UI) for intensity 0.5–2.0 in steps of 0.1; (iv) a submit `<button>` showing a spinner when loading. Style with Tailwind. The party selection must be prominent — it determines the entire analysis direction. [3 hrs]

□ **Task:** Add five preset shock buttons below the textarea. Each button pre-fills the event field with a template string. Presets: (i) Security: “An assassination attempt is made on the Democratic candidate”; (ii) Geopolitical: “The US enters a major armed conflict in the Middle East”; (iii) Moral/Scandal: “A major financial scandal involving the Republican candidate is revealed”; (iv) Electoral Surprise: “A major October Surprise leak dominates the news cycle”; (v) Economic: “A sudden recession is declared in the final month of the campaign”. Style each button with its shock category color. [2 hrs]

□ **Task:** Implement the error display: below the submit button, show a styled red alert box when `error` is non-null. The message should be user-friendly (“The estimation service is

unavailable. Please try again.”) rather than exposing the raw API error. Log the raw error to the browser console for debugging. [1 hrs]

Day 3 (Wed — Prediction Panel: ShockNarrative & CoalitionChart)

□ **Task:** Implement `webapp/components/ShockNarrative.tsx`. Props: `deltaBins: Record<string, string>`, `party: Party`. This component renders the plain-language **prediction** — what the model expects will happen to each voter group after the shock, in words the user can read without knowing what a delta bin is.

Logic: map each `delta_bin` token to a phrase pair (direction + magnitude): `strong_neg` → “significant loss among”, `mild_pos` → “modest gain with”, `neutral` → “no meaningful change among”, etc. Group blocs by direction (gaining vs. losing) and render two sentences:

- “This shock is predicted to **hurt** the [party] coalition’s standing with: [Evangelicals (significant), Catholics (mild)].”
- “And **help** with: [Latino voters (moderate), Women (mild)].”

If all blocs are neutral, render: “The model predicts minimal coalition impact from this event.” The narrative populates immediately when the `event: deltas` SSE event fires — the user sees the prediction in words before the charts finish rendering. [3 hrs]

□ **Task:** Implement `webapp/components/CoalitionChart.tsx` — the single unified chart that shows both the prediction and the rebalance in one view. Props: `baseline: Record<string, number>` (pre-shock loyalty μ_i), `shifted: Record<string, number>` (post-shock loyalty $\tilde{\mu}_i$ — the **prediction**), `rebalanced: Record<string, number> | null` (optimizer coalition weights $\tilde{w}_i$ — the **rebalance**; null until the `event: equilibrium` SSE event fires), `feasible: boolean`, `party: Party`.

Visual design — three series on one horizontal bar chart:

- **Opaque bars (prediction):** post-shock loyalty $\tilde{\mu}_i$, rendered at full opacity. Color: blue for Democrat, red for Republican. This is what the model says *will happen* to each group’s loyalty after the shock.
- **Translucent bars (rebalance):** optimizer coalition weight $\tilde{w}_i$, rendered at 35% opacity (`fillOpacity=0.35`) on top of the opaque bars using the same color. This is what the model says the coalition *should look like* to still win. Absent until `event: equilibrium` fires; the opaque bars are visible and readable while the rebalance is loading.
- **Reference tick marks:** a vertical `ReferenceLine` at μ_i (pre-shock baseline) per bloc, rendered as a thin gray tick, so users can see how far the shock moved loyalty from the starting point.

Why same axis works: both $\tilde{\mu}_i$ and $\tilde{w}_i$ are in $[0, 1]$. $\tilde{\mu}_i$ is within-bloc loyalty (typically 0.15–0.90). $\tilde{w}_i$ is coalition weight (typically 0.03–0.30). They occupy different ranges and are visually distinct. A clear legend is required: “— Prediction (loyalty shift)” and “— Rebalance (coalition weight, translucent)”. The tooltip on hover must label them explicitly with different units.

Sorting: sort blocs by `Math.abs(shifted[bloc] - baseline[bloc])` descending so the most-moved blocs appear at the top.

Labels: on each opaque bar, show the signed loyalty delta (+8pp, -3pp) in green/red. On each translucent bar, show the coalition weight as a percentage (14%).

Feasibility: if `feasible=false` and `rebalanced` is non-null, replace the translucent bars with a striped red pattern and add a banner: “No feasible coalition path under this shock.”
[5 hrs]

□ **Task:** Add a section header above `CoalitionChart` with two sub-labels that map to the two visual layers: “**What will happen**” (opaque, bold) and “**Most likely rebalance**” (translucent, muted) — styled as a color-coded legend inline with the header so users understand what each layer means before reading the chart. Below the chart, add a summary line: “Equilibrium status: **MET / NOT MET**” in green/red, plus the gap formatted as “+3.2 pp above target”.
[1 hrs]

Day 4 (Thu — WinGauge, SSE Streaming & Audit Log)

□ **Task:** Implement `webapp/components/WinGauge.tsx`. Props: `winProbability: number`, `percentiles: Record<string, number[]>`. Render a semicircular SVG gauge (viewBox 0 0 200 110). Draw the arc as three colored segments: red (0–45%), yellow (45–55%), green (55–100%). Draw a needle rotating from 0° (left) to 180° (right) based on `winProbability`. Animate the needle with a CSS transition on value change (`transition: transform 0.8s ease-out`).
[2 hrs]

□ **Task:** Add a percentile distribution strip below the gauge: a compact horizontal box-plot showing p5, p25, p50, p75, p95 per bloc from `SimulationData.percentiles`. Use a small `Recharts ComposedChart` with a `Bar` for the IQR and `ErrorBar` for p5–p95 whiskers. [1 hrs]

□ **Task:** Add the **market prior display** to the `WinGauge` component. After the model win probability renders, call `GET /api/market-prior?party=&event=` (new FastAPI route that queries Polymarket and Manifold for the nearest relevant contract via `electoral/markets/aggregator.py`). Display the volume-weighted result as “Market consensus: **X%**” in a muted gray style directly below the model estimate. If no active contract is found, show “No active market” in italics. The juxtaposition is the key epistemic display: the market aggregates all current public information; the model shows how the coalition structure shifts under the hypothetical. Add disclaimer: “Model: $N = 10,000$ simulations. Market: live Polymarket/Manifold prices.”
[2 hrs]

□ **Task:** Implement **SSE streaming** on the FastAPI backend. Add `GET /estimate/stream` using `StreamingResponse` with `media_type="text/event-stream"`. The generator yields three events: `event: deltas` (LLM output), `event: equilibrium` (optimizer), `event: simulation` (Monte Carlo). Wrap the optimizer and Monte Carlo calls correctly for their respective concurrency models:

- **CVXPY optimizer** — use `ProcessPoolExecutor`: `loop.run_in_executor(process_pool, solve_rebalanced, ...)`. CVXPY’s underlying C solvers (SCS/ECOS/OSQP) are not thread-safe and will segfault under `ThreadPoolExecutor`. Passing `None` defaults to a thread pool — never pass `None`. Instantiate `ProcessPoolExecutor(max_workers=1)` once at startup and reuse it.
- **Monte Carlo** (pure NumPy) — use `ThreadPoolExecutor`: `loop.run_in_executor(thread_pool, run_simulations, ...)`. NumPy releases the GIL during array operations; a thread pool is safe and lower-overhead than spawning a new process.

Each stage starts as soon as the previous one emits its SSE event. Expected latency improvement: ~30% on total round-trip. Update `webapp/lib/api.ts`: add `estimateShockStream(event, intensity, onDeltas, onEquilibrium, onSimulation)` that opens an `EventSource`, calls each callback as its event fires, and closes on `[DONE]`. [3 hrs]

□ **Task:** Implement **audit logging**. Create `electoral/api/audit.py`: `AuditLogger` class connecting to `data/audit.duckdb`. On first connect, run: `CREATE TABLE IF NOT EXISTS estimates (id INTEGER PRIMARY KEY, timestamp TIMESTAMP, event_text VARCHAR, intensity FLOAT, deltas_json VARCHAR, feasible BOOLEAN, target_met BOOLEAN, win_prob FLOAT, llm_ms INTEGER, optimizer_ms INTEGER, montecarlo_ms INTEGER, backend VARCHAR)`. Implement `log_estimate(...)` inserting one row per call. Wire into both `/estimate` and `/estimate/stream` endpoints. Add `GET /api/audit?limit=100` returning the most recent rows as JSON. Verify: call the endpoint 5 times, query DuckDB directly, confirm 5 rows exist. [2 hrs]

Day 5 (Fri — Polish, Deployment & PR)

□ **Task:** Assemble `webapp/app/page.tsx`: two-column layout on desktop, single-column on mobile. Add a sticky header with project name, a one-sentence description, and a link to the methodology document. Add the “How it works” expandable (`@radix-ui/react-collapsible`) and the research disclaimer. Test on iPhone viewport (375px). [2 hrs]

□ **Task:** Add a “Share this result” button that serializes `party`, `event`, and `intensity` as URL query params and copies to clipboard. Test that the URL pre-fills the full form including party selection on reload. [1 hrs]

□ **Task:** Deploy: `modal deploy deploy/modal_app.py`. Connect Vercel to the GitHub repo, set `NEXT_PUBLIC_API_URL` to the Modal URL. Confirm the full public end-to-end flow works for both party modes. Scaffold `deploy/hpc_vllm.sh` and `deploy/backend_router.py` (leave deactivated). [2 hrs]

□ **Task:** Weekly PR: `feature/week7-webapp`. Include: three public app screenshots (empty

state, Democrat Ayatollah result, Republican result, mobile layout) and the live Modal + Vercel URLs. Tag “demo-ready”. **Analyst dashboard is Week 8 Days 1–2; do not attempt it here.** [1 hrs]

☐ **Task:** Supervision check-in with Prof. Espinosa: live demo via public URL. Walk through the Ayatollah shock in both party modes. Collect domain feedback on result framing and disclaimer wording for use in Week 8 paper writing. [2 hrs]

8 Week 7 — Analyst Dashboard, Hardening & Deliverables (40 hrs)

Goal. By Friday, the analyst dashboard is live, all integration tests pass, the pipeline is replication-ready, and the paper and poster are complete. The dashboard was moved here from Week 7 to give it realistic time: SSE wiring, DuckDB, password auth, and six data-backed panels is a two-day build, not one.

End condition.

- ☐ Analyst dashboard live at `/dashboard` with all six panels.
- ☐ All six integration tests pass on a clean checkout.
- ☐ Replication package complete and documented.
- ☐ `CONTINUOUS_UPDATE.md` written; `just continuous` runs clean.
- ☐ Research paper draft submitted to SRP committee — must include: (i) formal quasiconvexity proof for the DQCP objective; (ii) explicit additive independence assumption statement; (iii) additive vs. raked model comparison on held-out cycles; (iv) MMD + PCA + PCD synthetic data validation report.
- ☐ Poster submitted to SRP.
- ☐ Final PR merged and tagged `v1.0`.

Day 1 (Mon — Analyst Dashboard: Auth, Backend Routes & Panels 1–3)

☐ **Task:** Implement dashboard authentication. Create `webapp/app/dashboard/page.tsx`: render a login form when no valid `dashboard_session` cookie exists; on submit POST to `/api/dashboard/auth` which compares against the `DASHBOARD_PASSWORD` env var and sets an 8-hour session cookie. Implement `webapp/app/api/dashboard/auth/route.ts` as the Next.js route handler. On success, render the dashboard shell with a `<DashboardNav>` sidebar (links to each panel, logout button). [2 hrs]

☐ **Task:** Implement the three FastAPI backend routes for dashboard data and wire them in `shock_endpoint.py`. GET `/api/coverage`: load the `VoterPanelData` artifact and return a JSON matrix of per-cell quality ("`multi-source`", "`single-source`", "`imputed`", "`missing`") with raw `vote_share` and sources. GET `/api/sentiment-dist?category=`: load `SentimentData` and return per-bloc elasticity score arrays, optionally filtered by shock category. GET `/api/bio-coverage`: return per-shock bio classifier coverage counts (keyword, `SetFit`, fallback). [2 hrs]

☐ **Task:** Implement **Panel 1 — Data Coverage Matrix** in `CoverageMatrix.tsx`: a `Recharts ScatterChart` rendered as a heatmap (bloc × cycle), colored dark blue (multi-source) through light blue (single) through gray (imputed) to white (missing). Tooltip shows raw `vote_share` and sources. [2 hrs]

□ **Task:** Implement **Panel 2 — RoBERTa Score Distributions:** a `ComposedChart` showing min, p25, median, p75, max per bloc across all shocks; shock category dropdown re-fetches with `?category=`. Implement **Panel 3 — Bio Classifier Coverage:** stacked `BarChart`, one column per shock, three segments (keyword / SetFit / fallback). Both panels share a two-column grid row. [2 hrs]

Day 2 (Tue — Analyst Dashboard: Backend Routes & Panels 4-6)

□ **Task:** Implement the three remaining FastAPI backend routes. `GET /api/training-logs:` read HPC training log files from `rawdata/hpc_logs/` (synced via Syncthing from HPC) and return per-epoch train/val loss arrays per run ID. `GET /api/convergence:` load the most recent `SimulationData` artifact and return win probability at $N = 1,000, 5,000, 10,000$ with p5/p95 bands. `GET /api/audit?limit=100&search=:` query `data/audit.duckdb` with an optional `WHERE event_text LIKE %search%` clause; return rows with all columns including party. [2 hrs]

□ **Task:** Implement **Panel 4 — Fine-Tuning Loss Curves:** Recharts `LineChart` with train (solid) and val (dashed) series, horizontal reference line at the 2020 MAE, run-selector dropdown. Implement **Panel 5 — Monte Carlo Convergence:** `AreaChart` with p5/p95 shaded band, dashed final estimate line, ± 0.005 tolerance band, Pass/Fail badge. [2 hrs]

□ **Task:** Implement **Panel 6 — Audit Log Query Interface:** sortable table with columns timestamp, event (40-char truncated), party, intensity, win probability, feasibility, total latency. Free-text search box. Row-expand shows the full per-bloc delta vector as a mini `CoalitionChart` and per-stage latency breakdown. [2 hrs]

□ **Task:** Assemble the full dashboard layout: two-column grid for Panels 1–2, full-width for Panels 3–6. Add page header “Electoral Equilibrium — Analyst Dashboard” with live estimate count from DuckDB. Test all six panels with real pipeline data. Verify panel filters work. [1 hrs]

Day 3 (Wed — Feedback, Integration Tests & Replication Package)

□ **Task:** Implement any domain feedback from Prof. Espinosa’s Week 7 supervision check-in. Common changes: bloc display name corrections, chart axis relabeling, disclaimer rewording. Re-run the manual end-to-end test after each change. Commit as `fix: incorporate-supervision-feedback`. [2 hrs]

□ **Task:** Write `tests/test_integration_end_to_end.py`: run the full pipeline on the smoke config from a clean state. Assert all stage artifact files exist at the expected paths, parse into their typed payload classes without error, and pass `validate()`. Assert the artifact envelope contains `stage`, `run_key`, `metadata`, `data` keys. [2 hrs]

□ **Task:** Write `tests/test_determinism_full_run.py`: run the pipeline twice with the same config and seed writing to two separate output directories. For each stage, load both artifacts, parse the `data` payload dict, and assert field-by-field equality for all non-path fields. Log any non-deterministic fields found. [2 hrs]

□ **Task:** Write `tests/test_artifact_schemas.py`: for each artifact produced by the smoke run, assert the envelope keys exist with correct types (`stage` is str, `metadata` is dict, etc.), and assert all required payload fields declared in the artifact classes are present in the `data` dict. [2 hrs]

□ **Task:** Run the full test suite on a clean virtual environment from a fresh `git clone`. Fix any import errors, missing `__init__.py` files, hard-coded absolute paths, or undeclared dependencies until the full suite passes on a clean machine state. [2 hrs]

□ **Task:** Run the complete pipeline on real data with `configs/paper_baseline.json`. Record total runtime per stage. Commit all stage artifacts to `artifacts/paper_baseline/`. Tag `v1.0-paper-baseline`. These artifacts must not be modified after this point — they are the paper’s ground truth. [2 hrs]

□ **Task:** Write `README.md` “How to Reproduce This Run” section and `scripts/verify_artifacts.py`. `README` must include exact Python version, step-by-step commands, HPC setup, expected runtimes, and how to verify outputs. The script loads `paper-baseline` and fresh artifacts side-by-side and prints a pass/fail report. [2 hrs]

□ **Task:** Write the **Methodology** section (target: 1,800 words). Structure: (1) *Data Sources* — describe the five survey sources and the social media collection across four platforms; introduce the baseline-adjustment procedure for selection bias. (2) *Voter Equilibrium Framework* — state the portfolio analogy formally with the QP from the development plan; define $V_{eq} = 0.535$ and explain the 3% margin-of-error rationale. (3) *Two-Stage NLP Pipeline* — describe RoBERTa as feature engineering layer, explain the fine-tuning dataset construction including social media elasticity features, state the QLoRA hyperparameters in a table. (4) *Constrained Decoding* — one paragraph explaining why standard sampling is insufficient for the web app and how `outlines` guarantees schema validity. [4 hrs]

□ **Task:** Write the **Results** section (target: 1,200 words). Structure: (1) *Baseline Portfolio* — report the optimized bloc weights and cross-validate against Prof. Espinosa’s documented thresholds (Evangelical 24%, etc.); include the LOCO validation accuracy table. (2) *Baseline Comparison* — present the three-way MAE table (news-only, social-only, unified LLM); highlight which blocs benefit most from the social media signal. (3) *Shock Simulations* — present win probability and bloc-weight shifts for 3–4 representative historical shocks (one per category); include the sensitivity analysis plot for the Ayatollah event. (4) *Platform Correlations* — one paragraph reporting the per-platform correlation with 14-day lagged

favorability polls.

[4 hrs]

Day 4 (Thu — Paper Completion & Poster)

□ **Task:** Write the **Introduction** (target: 600 words). Open with a concrete example: the Ayatollah shock and its measurable effect on evangelical favorability. State the three gaps in existing work: (i) polling aggregators are static; (ii) advanced systems are enterprise-only; (iii) religious and demographic granularity is underused. State the three contributions: (i) the voter equilibrium framework; (ii) the two-stage RoBERTa-to-LLM pipeline with social media augmentation; (iii) the public web application. End with a roadmap of the paper. [2 hrs]

□ **Task:** Write the **Literature Review** (target: 700 words). Four subsections: (1) *Electoral Forecasting* — FiveThirtyEight, Economist model, prediction markets; their shared limitation as polling aggregators. (2) *Religion and Race in Presidential Politics* — cite Prof. Espinosa’s series (all five books, key articles); the Evangelical threshold, Latino Pentecostal patterns, Catholic swing behavior. (3) *NLP in Political Science* — sentiment analysis for event detection, existing social media political forecasting work. (4) *Portfolio Theory in Non-Financial Contexts* — precedents for applying mean-variance optimization outside finance. [2 hrs]

□ **Task:** Write the **Discussion and Limitations** (500 words) and **Conclusion** (300 words). Discussion must address: (i) platform selection bias and the baseline-adjustment mitigation; (ii) small fine-tuning dataset and why QLoRA regularizes against overfitting; (iii) the model should display uncertainty prominently and must not be used for campaign strategy. Conclusion: summarize contributions, state the 2026 midterm validation plan, note the open-source release. [2 hrs]

□ **Task:** Design the conference poster ($48'' \times 36''$) in Canva. Sections: (i) title block with authors and CMC; (ii) pipeline diagram (the full flow from data to web app); (iii) the baseline comparison table (roberta_news_only vs. roberta_social_only vs. llm_unified MAE per bloc); (iv) a CoalitionChart screenshot for the Ayatollah event; (v) the win gauge for one Electoral Surprise shock; (vi) a dashboard screenshot (Panel 1 coverage matrix or Panel 6 audit log); (vii) QR code linking to the live Modal + Vercel URL. Export as PDF and PNG. Share with Prof. Espinosa for any domain input before finalizing. [2 hrs]

Day 5 (Fri — Deployment, Final Submission & Retrospective)

□ **Task:** Incorporate any domain feedback from the Week 8 supervision check-in. Finalize paper and poster as PDFs. Submit both to the SRP committee per their submission instructions. [2 hrs]

□ **Task:** Verify the Modal + Vercel deployment is stable: run 20 back-to-back inference calls from the public URL, confirm all return valid JSON, confirm mean latency is under 5 seconds

(Modal cold start included). If latency is too high, switch Modal to a `keep_warm=1` container to eliminate cold starts. Document final latency numbers in `docs/latency_benchmark.md`. [1 hrs]

□ **Task:** Verify the HPC scaffold: `sbatch deploy/hpc_vllm.sh` on the HPC and confirm the vLLM server starts and the health check passes on the HPC-internal URL. Do not expose externally yet — this is a dry run to confirm the scaffold works. Log the internal URL and SLURM job ID in `docs/hpc_deployment_notes.md` for future activation. [1 hrs]

□ **Task:** Pin all dependency versions: `run pip freeze > requirements.txt` in the activated venv. Update `pyproject.toml` to match. Commit. Confirm a fresh install from `requirements.txt` passes the full test suite. [1 hrs]

□ **Task:** Write `CONTINUOUS_UPDATE.md` — the post-SRP maintenance specification. This document ensures the model does not become stale after the project ends. Required sections:

1. **Nightly data ingestion.** A launchd plist on the **Intel Mac** triggers a Prefect flow at 2am using `pipeline_mode="continuous"`: runs `BlueSkyCollector` and `ApifyCollector` for any shock events in the last 7 days; runs `sample_archives.py` on any new archive data; runs `clean_with_llm.py` on the new sample; scores new posts via RoBERTa; appends to `data/finetune/new_events.jsonl`. The Intel Mac is the right machine for this: it is always on, already running collectors, and can submit HPC fine-tuning jobs when the threshold is reached without any human intervention. Include the launchd plist and the `just continuous` Justfile target.
2. **Fine-tuning trigger.** When `new_events.jsonl` reaches 10+ new shock events, submit a new HPC fine-tuning job. Archive the current adapter with a timestamp before replacing it.
3. **Voter panel refresh.** Annual check: ARDA, GSS, Pew, and Gallup all release updated data. When new data is available, re-run the Week 2 pipeline. The win threshold $V_{eq}^{(P)}$ and constraint bounds update automatically via `build_constraint_spec`.
4. **Prediction market calibration.** Quarterly: query `audit.duckdb` for resolved shock events; compare `win_probability` predictions against actual market outcomes. Document any systematic divergence and adjust fine-tuning example weights accordingly.
5. **How to trigger a manual run.** Commands for `just continuous`, `just train`, and `just deploy`.

[2 hrs]

□ **Task:** Create `configs/continuous.json`: the configuration file for nightly incremental runs. Set `pipeline_mode: "continuous"`, `lookback_days: 7` (only collect shocks from the last 7 days), `skip_voter_panel_rebuild: true` (voter panel only rebuilt annually), `finetune_trigger_threshold: 10` (trigger retraining after 10 new shock events). Add `just continuous` to the Justfile: `prefect run pipeline.py --config`

`configs/continuous.json`.

[1 hrs]

□ **Task:** Final PR: `feature/week8-hardening-deliverables`. This PR must include: (i) all integration and hardening tests, (ii) the frozen paper-baseline artifacts, (iii) `requirements.txt` with pinned versions, (iv) `scripts/verify_artifacts.py`, (v) the complete `README.md`, (vi) the `Justfile` with all targets verified to work on a clean checkout including `just continuous`, (vii) `DOMAIN.md` reviewed and complete, (viii) `CONTINUOUS_UPDATE.md` complete. Merge to `main`. Tag as `v1.0`. [1 hrs]

□ **Task:** Write `docs/retrospective.md`: a one-page honest assessment covering (i) what went as planned, (ii) what took longer than expected and why (likely: API approvals, HPC queue wait times, data cleaning), (iii) what you would do differently with a 10-week timeline, (iv) the concrete publication plan (target journal, submission timeline), (v) the 2026 midterm validation strategy (which shocks to monitor, how to update the model with new data). [2 hrs]

□ **Task:** Final supervision check-in with Prof. Espinosa: project wrap-up. Share the submitted paper and poster. Get domain sign-off on the framing and discuss the 2026 midterm validation plan. Agree on publication timeline and the 2–3 most important follow-up directions. [2 hrs]

AI Research Assistant — Quick Reference

Setup: OpenClaw on Raspberry Pi 5 → Telegram/WhatsApp interface → Gemini Pro (default) or Claude Sonnet (@claude prefix). Syncthing shared folder across the three local machines (M5, Intel Mac, Pi).

Machine Allocation at a Glance

Machine	Always running	When needed
M5 MacBook Pro	Development, Claude sessions	MLX fine-tune experiments
Intel MacBook	Bluesky + Apify collectors	FastAPI endpoint in Week 7
	(24/7); nightly pipeline (2am launchd)	
Intel Mac	News scraper (was Windows role), collectors	launchd 2am nightly pipeline
Pi 5 + AI HAT	Bio classifier NPU, OpenClaw	—
HPC	—	Fine-tuning, scoring, Monte Carlo

When to use Gemini vs. Claude

Task	Model	Example message
Summarize a paper	Gemini	“Summarize arxiv.org/abs/2401.00000”
Morning project brief	Gemini	Automated 8am cron
Quick code question	Gemini	“What does <code>scale_pos_weight</code> do in XG-Boost?”
Find references	Gemini	“Find 3 papers on NLP and electoral forecasting”
Draft a paragraph	Gemini	“Draft an intro paragraph on portfolio theory”
Review pipeline logic	Claude	“@claude review kernels/shock.py”
Debug test failure	Claude	“@claude debug this pytest output: [paste]”
Write or fix LaTeX	Claude	“@claude fix the overflows in section 4”
Architecture decision	Claude	“@claude should I use Ridge or Lasso for the elasticity regression?”
Full session work	Claude	Open claude.ai directly

Useful commands

Command	Action
<code>openclaw doctor --fix</code>	Validate config and fix common issues
<code>openclaw skills install <name></code>	Install a skill from ClawHub
<code>openclaw logs</code>	View recent agent activity
<code>openclaw config edit</code>	Open config in editor
<code>@claude <message></code>	Route this message to Claude Sonnet
<code>@gemini <message></code>	Explicitly route to Gemini Pro

Cost guardrails — spend only where it matters

Service	Cost	Programme	Action
CMC HPC	Free	CMC student allocation	Use aggressively
Modal GPU inference	~\$0.0003/call	\$30/mo free credit	~100k calls free
Vercel frontend	Free	Hobby tier	No action needed
GitHub	Free	Student Developer Pack	Apply at education.github.com
Namecheap domain	Free	via GitHub Student Pack	Claim via Student Pack
Gemini Pro	Free	Google AI Studio API	60 RPM, 1k req/day
Bluesky AT Protocol	Free	Open source	Primary social; full firehose
Apify X Scraper	Free	Free tier	Secondary live; 500 results/run
Meta Content Library	Free	Academic research	Apply Week 0
Claude Pro Max	Already paid	Existing subscription	Reserve for hard problems

Expected total cost during development: **\$0–\$5/month.**

Set a \$10/month hard cap on the Modal API at modal.com/settings/billing to prevent unexpected charges if demo traffic spikes. Do not use `keep_warm` on the Modal container during development — cold starts cost nothing and the \$30 free credit is sufficient for all testing. Reserve paid Modal time for the live demo day only.